



M256 Unit 8

UNDERGRADUATE COMPUTING

Software development with Java



Designs in action

Unit 8

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007. Second edition 2008.

Copyright © 2007, 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 5481 0

CONTENTS

1	Introduction	5
1.1	Designs and working scenarios	5
1.2	The aims of this unit	5
1.3	Studying this unit	5
2	Returning information	7
2.1	The DVD Library System – setting the scene	7
2.2	The <i>List Films</i> use case	9
2.3	The design in action	17
3	Removing objects and links	20
3.1	Removing links	20
3.2	The <i>Return DVD</i> use case	23
4	Creating objects and links	32
4.1	The <i>Borrow DVD</i> use case	32
5	Derived attributes and the <i>Get Number Available</i> use case	39
5.1	Derived attributes	39
5.2	The <i>Get Number Available</i> use case	40
5.3	Putting the designs to work	42
6	Summary	46
	Glossary	47
	Index	48

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

The main purpose of this unit is to give you practice in what you have learnt in *Units 6* and *7*. You have already seen plenty of designs, but you may not yet be very confident, or fluent, in their creation and use, or in comparing different possible designs. In this unit you will gain more experience, including carrying out some experiments with sample software we have provided, and by the end of the unit you should feel much more at home with creating, interpreting and comparing designs.

1.1 Designs and working scenarios

In this unit you will be working with the DVD Library System that was introduced in *Unit 3*. You will apply the design approach first introduced in *Unit 6* to the four use cases of this system, creating, for each, an initial design based on a typical scenario – that is, an example of the use case involving a typical set of objects. Along the way you will learn what is involved in removing objects and links. You will also explore **working scenarios** – code that sets up the objects of a typical scenario and then realises the design as a ‘proof of concept’. These working scenarios show how the design might be implemented, and demonstrate that the object interaction ‘works’. It is for this reason that this unit is titled *Designs in action*.

Working scenarios can also be considered to be the end points of initial iterations in the development of the full system. Considered in this light, the next stage in the development process would be to expand the designs to deal with different scenarios and communication with the user interface, eventually leading to a fully operational implementation of the DVD Library System. However, the main aim of studying working scenarios in this unit is to give you a greater understanding of the consequences of design decisions.

This unit does not consider the issues involved in developing and writing the code for a given design – these matters are dealt with in *Units 9* and *10*.

1.2 The aims of this unit

This unit aims to:

- ▶ give you more practice at developing walk-throughs and sequence diagrams, applying the ideas learnt in earlier units (Sections 2, 3, 4 and 5);
- ▶ develop your design skills, to deal with removing objects and links (Section 3);
- ▶ allow you to experiment with code that demonstrates designs for use cases in action, i.e. working scenarios (Sections 2 and 5).

1.3 Studying this unit

This unit contains a number of SAQs and paper-based exercises, which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on.

This unit is approximately the same length as *Unit 7*. It introduces few new concepts and you may find that it takes less than the time indicated on the *Study Calendar* to complete.

Sections 2, 3 and 4 are the most substantial sections, requiring approximately equal study time, while Section 5 is a relatively short section

Sections 2 and 5 both contain practical exercises – these mostly involve running software and examining code that has already been written.

Throughout this unit you will need to refer to information about the DVD Library System, namely its requirements document and its initial structural model. These can be found in Sections 8 and 9 of the *Handbook*. We suggest you keep it by your side as you work through the unit.

2 Returning information

In this section you will participate in the creation of a design for one of the DVD Library System's use cases, *List Films* (labelled B in the requirements document given below). The section will conclude by analysing a working scenario: a short Java program that implements the design as skeleton Java code, and applies it in a particular scenario, thus showing the design 'in action'.

2.1 The DVD Library System – setting the scene

You first met this system in *Unit 3*, Subsection 3.2, where the relevant part of its requirements document was set out. For ease of reference, it is repeated below.

This extract can also be found in Section 8 of the *Handbook*.

System domain

The DVD Library holds DVDs of a number of films. It may have any number of copies of each film. The system should, for each film, hold the film's title; no two films have the same title.

A library member can borrow any DVD of a film that is not currently on loan. However, no member can have more than six DVDs on loan at once. A borrowed DVD must be returned within three days. When a library member returns a DVD it immediately becomes available for loan to other members.

Each DVD has a unique identifying number.

Each member of the library has a unique membership number; the system also holds each member's name.

Use cases

The DVD Library System will provide support to the library by enabling the library staff to do the following:

- A Get Number Available.** The staff member identifies the film. The system displays the number of copies available.
- B List Films.** The staff member identifies the member. The system displays a list of the film titles and return dates for the DVDs that the member has on loan.
- C Borrow DVD.** The staff member identifies the DVD and the member who is borrowing it. The system records the fact that this copy of the film is no longer available for loan, the fact that it is on loan to the member identified, and the date on which it was borrowed.
- D Return DVD.** The staff member identifies the DVD. The system records the fact that this copy of the film is now available for loan and that it is no longer on loan to the member who returned it.

From this requirements document was developed (in *Unit 3*) the following initial class diagram (Figure 1).

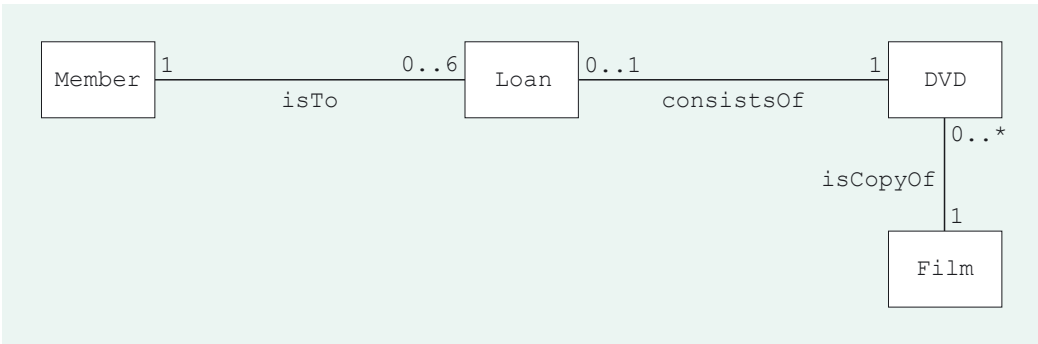


Figure 1 Initial class diagram for the DVD Library System

Here are the class descriptions and invariants.

Class descriptions

Class	DVD	A DVD of a film
Attributes		
	number	The unique identifying number of the DVD
Class	Film	A film
Attributes		
	title	The unique title of the film
	/numberAvailable	The number of copies of the film available for loan
Class	Loan	The loan of a DVD
Attributes		
	issueDate	The date of issue of the loan
	/returnDate	The date by which the loaned DVD must be returned.
Class	Member	A library member
Attributes		
	number	The unique membership number of the member
	name	The name of the member

Invariants

- 1 For each Loan object, the difference between the values of its issueDate and returnDate attributes is 3 days.
- 2 For any Film object, the value of its numberAvailable attribute is equal to the number of DVD objects it is linked to, which are not linked to a Loan object.

SAQ 1

What extra class needs to be added to those in Figure 1 when beginning the design process?

ANSWER.....

A coordinating class must be added to handle communication between the user interface and the core system.

This issue was discussed in Unit 6, Subsection 2.1, just prior to Figure 11.

In our design we have called the coordinating class `LibCoord` (Figure 2).

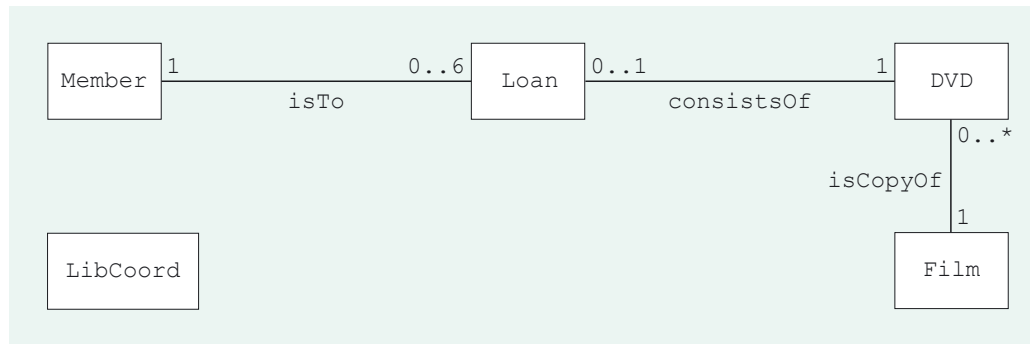


Figure 2 The class diagram including the coordinating class `LibCoord`

The `LibCoord` class has a single instance, the coordinating object, which we will call *library*. A typical use case begins with a coordinating message being sent to this object, which then handles all interaction with the core system.

Having set the scene, the next task is to begin constructing dynamic models for the use cases.

2.2 The *List Films* use case

Here is the description of the *List Films* use case.

The staff member identifies the member. The system displays a list of the film titles and return dates for the DVDs that the member has on loan.

This use case requires information to be obtained from the core system, but it does not change the state of the core system.

SAQ 2

In M256 when information is required by the user interface, what form do the results returned by the core system take?

ANSWER.....

The results returned to the user interface are in the form of the objects whose attributes contain the information that is required. It is up to the user interface to extract the required information from the objects by sending the appropriate getter messages.

The *List Films* use case is very similar to the *List Team's Patients* use case (labelled F) from the Hospital System, which you studied in *Unit 6*, Subsection 3.2, only this time you will find that four core classes are involved instead of three. This subsection will follow the example of the *List Team's Patients* use case quite closely and you may find it useful to refer to its dynamic model in Section 5 of the *Handbook* to remind yourself of the details.

Our analysis will follow the same five-stage approach that was introduced in *Unit 6*, Subsection 3.1.

- 1 Choose the objects for a typical scenario and illustrate with an object diagram or diagrams.
- 2 Draw a sequence diagram showing just the communication between the user interface and the coordinating object.
- 3 Specify a coordinating method.

This design approach is also set out in Section 2 of the *Handbook*.

- 4 Devise a walk-through and a sequence diagram (extending the one in step 2) for the chosen scenario.
- 5 Note the implications of the walk-through and sequence diagram for the structural model, and amend it if necessary.

In working through the *List Films* use case we shall indicate, with subheadings, which stage of the above approach has been reached.

Stage 1 – choose the objects for a typical scenario

The first task is to create an object diagram to illustrate a typical scenario of the use case. Exercise 1 asks you to consider what this should contain.

Exercise 1

What objects and links need to be shown in an object diagram that represents a single member and the films that the member has DVDs of out on loan?

Discussion.....

The diagram should show the *Member* object together with all the *Loan* objects linked to it, the *DVD* object linked to each of these *Loan* objects and the *Film* object linked to each *DVD* object. All the links between these objects should also be shown.

We suggest that, for practice, you stop and sketch an object diagram of your own before going on, but work from the one in Figure 3 for subsequent stages, so your answers will agree with ours. Remember that your object diagram should be based on the class diagram given in Figure 2.

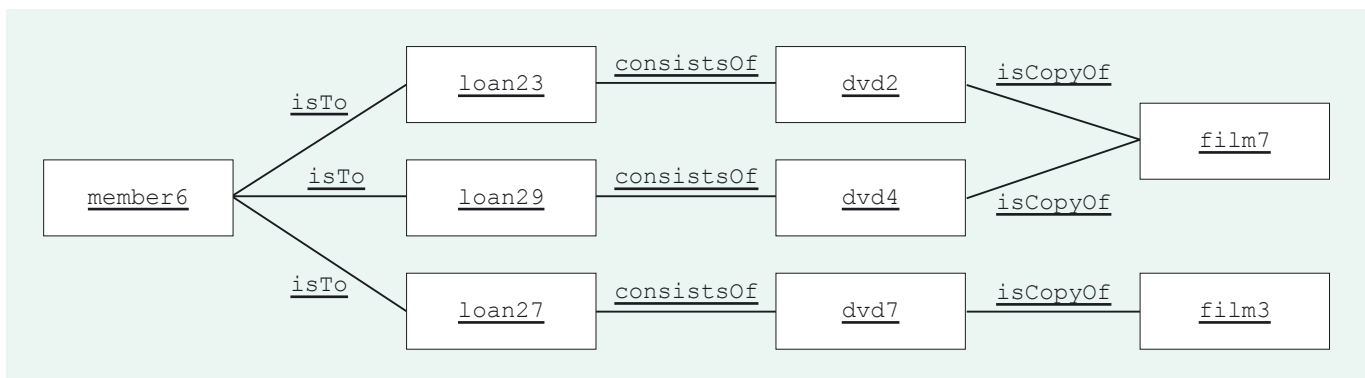


Figure 3 Object diagram for the *List Films* scenario

After identifying the objects for a typical scenario, it is now necessary to model the communication between the user interface and the coordinating object.

Stages 2 and 3 – draw a sequence diagram and specify a coordinating method

Exercise 2

- (a) Describe what kinds of object will need to be returned to the user interface by a coordinating method for *List Films*. (Consider what information the user interface needs to display.)

- (b) Draw a sequence diagram for *List Films* showing just the coordinating message being sent to the coordinating object and the appropriate collection being returned. You should assume a scenario based on Figure 3.
- (c) Write a specification of the coordinating method for *List Films*. You may find it useful to refer to the specification of the coordinating method for *List Team's Patients* in Section 5 of the *Handbook*.
- (d) Does the use case have any pre-conditions?

Discussion.....

- (a) A collection of pairs each of which consists of a `Loan` object and a `Film` object should be returned. (The user interface will then be able to find out the return date from each `Loan` object and the film title from each `Film` object.)
- (b) Our diagram is shown in Figure 4. The user interface passes in a `Member` object and gets back a collection of pairs of `Loan` and `Film` objects.
`getLoansAndFilms(member6)` is an appropriate message but any sensible name will do. The argument should be `member6` and the collection returned should correspond to Figure 3, with its elements being in any order.



Figure 4 Communication between the user interface and the coordinating object for *List Films*

- (c) The coordinating method specification should be on the following lines. (Note that this is very similar to the one for *List Team's Patients* in the Hospital System, `getPatientAndWards(aTeam)`, except that `Film` objects are not *directly* linked to the `Loan` object. As we did for *List Team's Patients*, we have chosen a `Map` object to hold the pairs of objects returned as the answer.)

```
Map<Loan, Film> getLoansAndFilms(Member aMember)
```

Post-condition: returns a map of pairs of `Loan` and `Film` objects, such that for each `Loan` object `aLoan` linked to `aMember`, the map contains the key-value pair (`aLoan`, `aFilm`) where `aFilm` is linked to the `DVD` object that is linked to `aLoan`.

- (d) No, *List Films* has no pre-conditions.

Stage 4 – devise a walk-through and extend the sequence diagram

The coordinating method specification establishes what the use case has to achieve. The next stage is to analyse how the specification will be realised as a series of individual actions within the core system. Notice that although the specification is

expressed in terms of *general* objects `aMember`, `aLoan` and so on, the walk-through and sequence diagram are based on the *particular* objects in the scenario under consideration.

Which object should be responsible for constructing the collection of `Loan` and `Film` pairs? There are two fairly obvious possibilities; one is the coordinating object, and the other is the `Member` object concerned.

SAQ 3

Suggest why it might be preferable to make the `Member` object responsible for constructing the collection of `Loan` and `Film` pairs, rather than the coordinating object.

ANSWER.....

One important reason involves cohesion. It is undesirable for the coordinating object to do too much, because that would produce a complex and overworked class that would be difficult to understand – it would show poor cohesion.

To improve cohesion, responsibilities should be distributed, with each responsibility allocated to the most appropriate object. Since *List Films* is concerned with the loans of a particular member, placing the responsibility with the `Member` object (which is the information expert in this case) is a natural choice. It is appropriate to expect this object to be able to provide the collection of `Loan` and `Film` objects corresponding to that member's loans.

You met the concept of an information expert in *Unit 7*, Subsection 2.2.

Here, in contrast to *Unit 6*, different designs are being considered at an early stage and the kinds of choice that were not considered until *Unit 7* for the Hospital System are already being made. In *Units 6* and *7* the aim was to introduce the techniques gradually and so initially only one design was considered for each use case. Now that you are aware that it may be necessary to choose between different designs, it is sensible to consider, or at least make a note of, any alternatives as they arise.

Having decided to allocate responsibility for constructing the collection to the `Member` object, the walk-through and sequence diagram can be constructed. The system is given the `Member` object, `member6`. So, the walk-through begins:

Given: the `Member` object, `member6`.

You will now need to refer to the object diagram in Figure 3.

Starting from `member6`, a collection of the relevant `Loan` and `Film` objects needs to be formed. Clearly the first step is to access the `Loan` objects linked via `isTo` to `member6`.

- 1 Access the collection of `Loan` objects linked to `member6`, which in this case is {`loan23`, `loan27`, `loan29`}.

Now, for each of these `Loan` objects, the corresponding `Film` object needs to be accessed. This can be done by first accessing the `DVD` object linked to the `Loan` object and using the `DVD` object to access the `Film` object. Each of these `Loan` and `Film` objects needs to be added to the collection, so first an empty collection will be set up:

- 2 Create a new empty collection `results`.

For each `Loan` object, `aLoan`, accessed in 1:

- 3 access `aDVD`, the `DVD` object linked to `aLoan`;
- 4 access `aFilm`, the `Film` object linked to `aDVD`;
- 5 add (`aLoan`, `aFilm`) to `results`.

Finally the collection has to be returned to the user interface. Putting this all together gives the following walk-through.

Given: the `Member` object, `member6`.

1 Access the collection of `Loan` objects linked to `member6`, which in this case is `{loan23, loan27, loan29}`.

2 Create a new empty collection `results`.

For each `Loan` object, `aLoan`, accessed in 1:

3 access `aDVD`, the `DVD` object linked to `aLoan`;

4 access `aFilm`, the `Film` object linked to `aDVD`;

5 add `(aLoan, aFilm)` to `results`.

6 Return `results`.

The sequence diagram has not yet been considered. Obviously step 1 will be done by `member6` (using its links with the `Loan` objects) and it has already been decided (SAQ 3) that the collection should be formed by `member6`. This gives the outline sequence diagram shown in Figure 5.

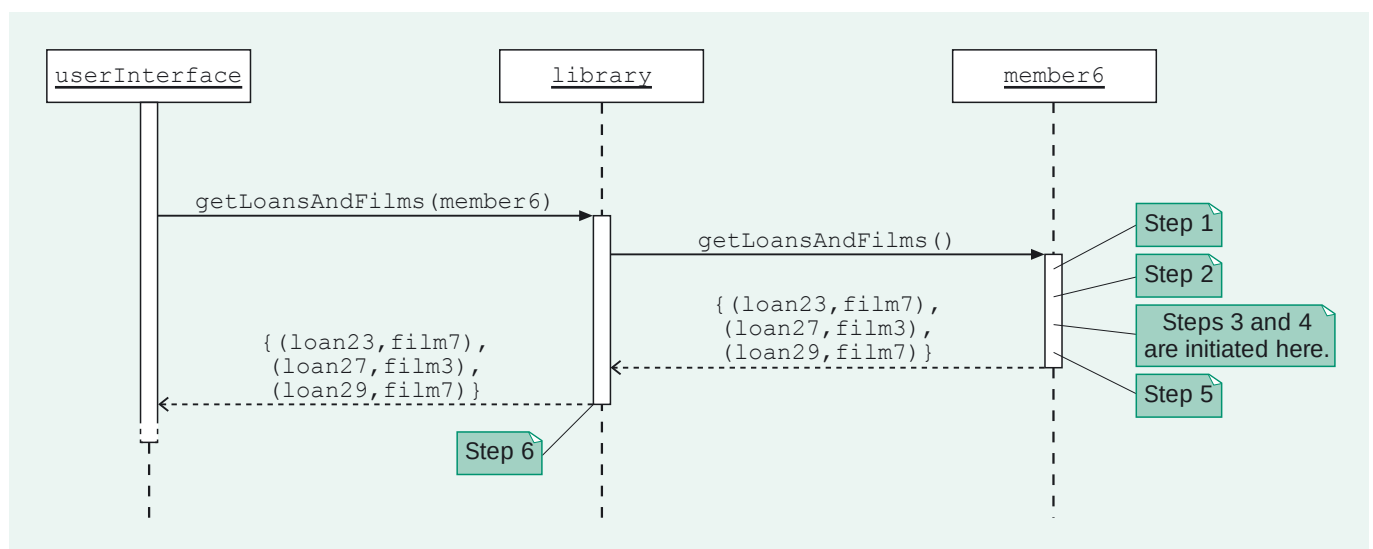


Figure 5 An outline sequence diagram for *List Films*

Having accessed a `Loan` object, there are two different designs for carrying out steps 3 and 4.

(i) `member6` accesses the `DVD` object.

In this case `member6` would send a message to the `Loan` object asking it to return the `DVD` object (step 3). `member6` can then directly ask the `DVD` object for its `Film` object (step 4).

(ii) `member6` does not access the `DVD` object.

Here `member6` sends a message to the `Loan` object asking it to return the required `Film` object. In this case it will be the `Loan` object that asks the `DVD` object for its `Film` object, which the `Loan` object then returns to `member6`.

Exercise 3

- For each of the designs listed as (i) and (ii), above, draw the corresponding sequence diagram in the case where the `Loan` object is `loan29`. Your diagrams should show just the objects `member6`, `loan29` and `dvd4`, and only the messages required for steps 3 and 4. Show on each diagram where steps 3 and 4 take place.
- Which of these sequence diagrams illustrates a fork and which illustrates a cascade?
- Give a reason for preferring the design listed as (ii).

Discussion.....

(a) For design (i), the sequence diagram is shown in Figure 6.

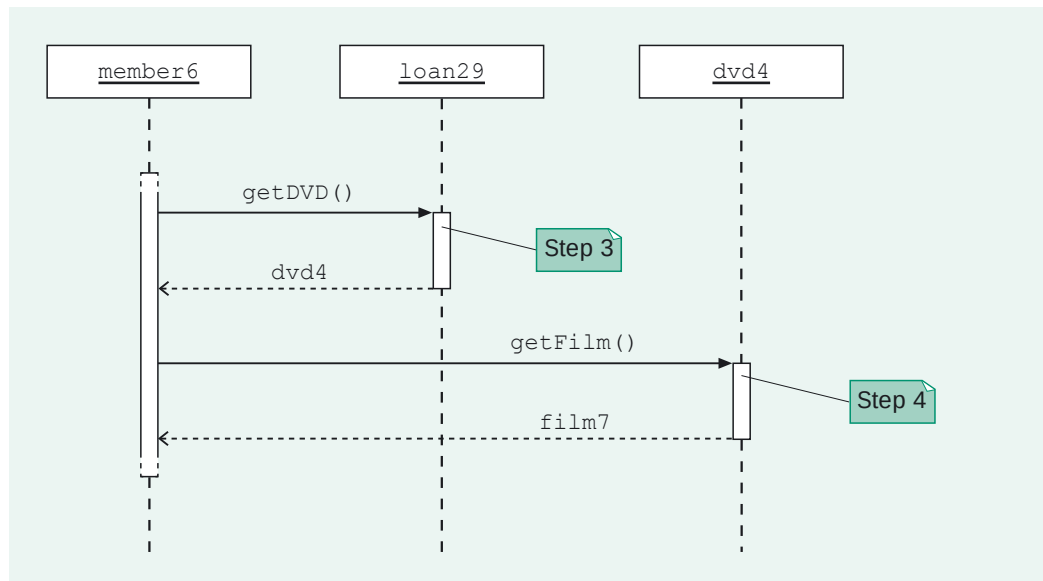


Figure 6 `member6` accesses `dvd4`

For design (ii), the sequence diagram is shown in Figure 7.

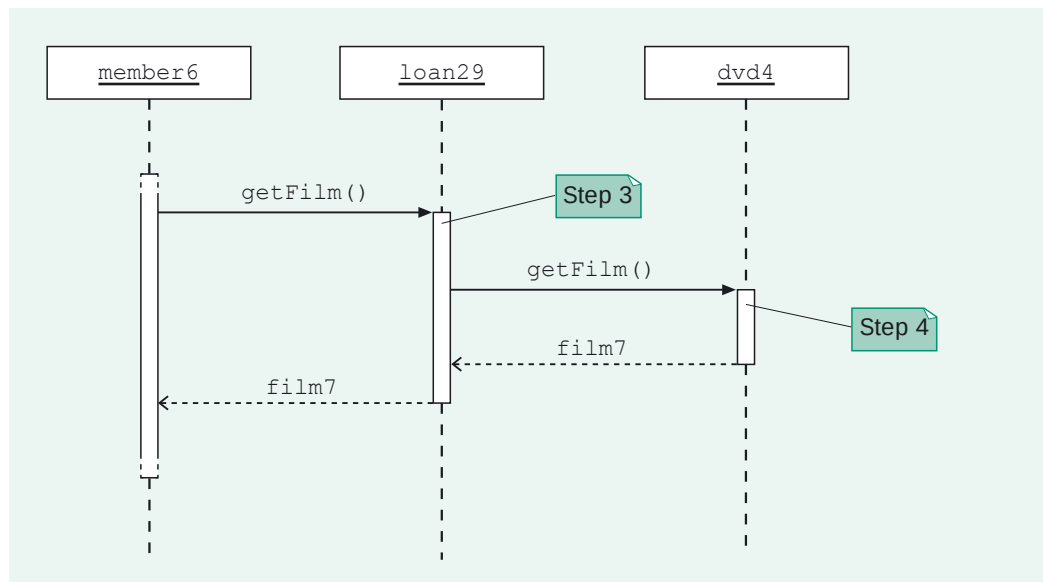


Figure 7 `member6` does not access `dvd4`

(b) The first diagram illustrates a fork. `member6` sends a message to the object that was returned as the answer from the `getDVD()` message.

The second diagram illustrates a cascade. `member6` delegates getting the `Film` object to `loan29`, which in turn passes the request onto `dvd4`.

(c) The second design is preferred as it avoids `member6` having to access and send a message to a `DVD` object to which it is not linked (which would tend to increase coupling). This design does, of course, require the `Loan` object (like the `DVD` object) to have a `getFilm()` method which the fork solution does not need; however getting the `Film` object seems an appropriate service for a `Loan` object to provide.

Putting all this together and using the cascade design for steps 3 and 4 gives the sequence diagram for our scenario shown in Figure 8.

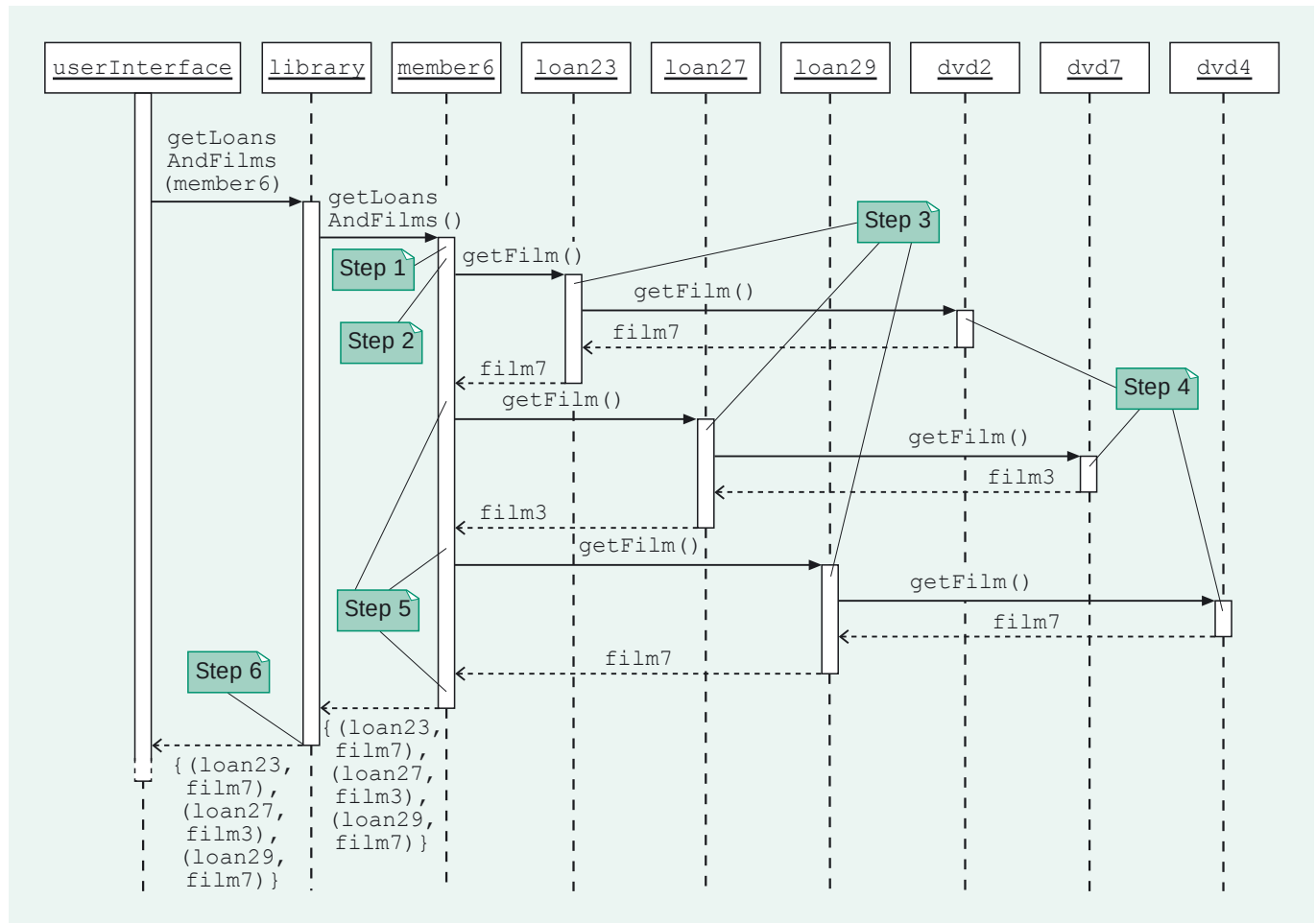


Figure 8 Sequence diagram for *List Films*

Stage 5 – amend the structural model

Exercise 4

- (a) What are the consequences of the design expressed by the dynamic model (i.e. the walk-through and the sequence diagram) for the navigability of the associations *isTo*, *consistsOf* and *isCopyOf*?

Redraw the class diagram for the DVD Library System, shown in Figure 2, with arrows to show these navigations.

- (b) Corresponding to these navigations, which classes have responsibilities for holding link references, and how are each of these responsibilities recorded in the emerging implementation model?

Discussion.....

- (a) Since, for example, *loan23*, *dvd2* and *film7* were reached in turn by following links from *member6*, it must be possible to navigate from *Member* to *Loan*, from *Loan* to *DVD* and from *DVD* to *Film*. These navigations are illustrated in Figure 9.

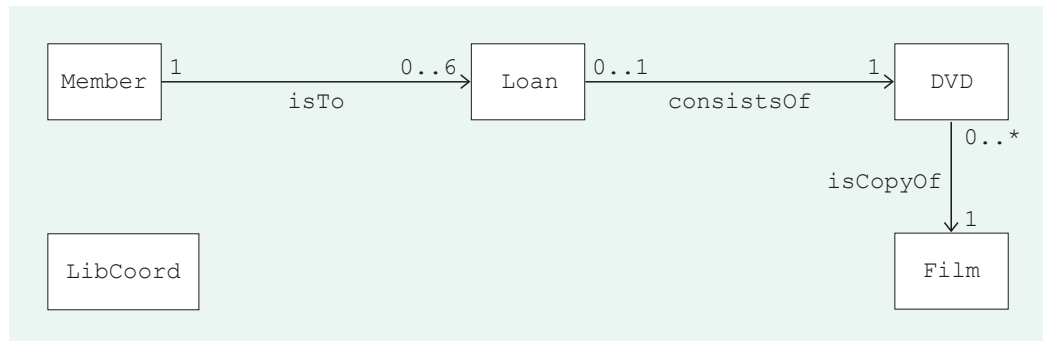


Figure 9 The class diagram with navigations added

(b) Member, Loan and DVD will have responsibilities for holding link references.

These are recorded as follows.

In the class Member:

Links

```
Collection<Loan> loans
```

References a collection of all the linked Loan objects.

In the class Loan:

Links

```
DVD dvd
```

References the linked DVD object.

In the class DVD:

Links

```
Film film
```

References the linked Film object.

In Exercise 4 you used the dynamic model to identify what navigations are needed for the design. In Exercise 5 you will use it to identify the methods that the design requires the classes involved to provide.

Exercise 5

- What methods does the design chosen for *List Films* require, and in what classes? (Give only the method name and any arguments.)
- The coordinating method has already been specified. Write a specification for each of the other methods identified in part (a).

Discussion.....

- The methods required are as follows:

In LibCoord: `getLoansAndFilms(aMember)`

In Member: `getLoansAndFilms()`

In Loan: `getFilm()`

In DVD: `getFilm()`

(b) The specification of these methods is as follows:

In the class `Member`:

```
Map<Loan, Film> getLoansAndFilms()
```

Post-condition: returns a map of pairs of `Loan` and `Film` objects such that, for each `Loan` object, `aLoan`, linked to the receiver, the map contains the key-value pair (`aLoan`, `aFilm`) where `aFilm` is linked to the `DVD` object that is linked to `aLoan`.

Notice that the description of what this method does is similar to that for the coordinating method, because the responsibility for carrying out the task of constructing the collection has been delegated to `Member`.

In the class `Loan`:

```
Film getFilm()
```

Post-condition: returns the `Film` object that is linked to the `DVD` object that is linked to the receiver.

In the class `DVD`:

```
Film getFilm()
```

Post-condition: returns the linked `Film` object.

2.3 The design in action

In *Units 6* and *7* we included specimen code for some of the designs to show possible implementations and to give you a feeling for how the design could be made to ‘work’ in Java. In this unit we go further and provide Java projects – working scenarios – which implement the designs devised for the use cases. These projects are designed to enable you to try out the design for a use case based on the scenario used in the design process. That is, the implementation will involve real objects corresponding to each of the example objects of the scenario: for example, there will be a `Member` object referenced by a variable `member6`, corresponding to the object of the scenario with identifier `member6`.

It is important to note that the implementations used in this unit are simple, ‘rough and ready’ pieces of code that provide a practical way of trying out the designs. In particular the implementations do not involve a user interface as such. In *Units 9* and *10* you will be looking in much more detail at what is involved in implementing a system and the choices that need to be made. There you will learn how to produce your own code from a given design; in this unit we will simply present you with the code.

You could consider this as completing a first iteration in an iterative approach to the design of the DVD Library System. Having designed the first use case, enough code to implement the design is mocked up so that it can be tried out for a simple scenario, enabling a better insight into the designs. Of course, running code in this way is not testing as such, although it does enable you to confirm in an informal way that the designs ‘work’.

In the next piece of practical work (in Section 4) we complete a second iteration by providing code corresponding to the designs of the other three use cases. Later iterations would go on to improve the designs by considering different scenarios and communication with a user interface.

The project you will use in Exercise 6, as with the later projects in this unit, has two packages. One, called `listfilms_core` (the core system package), contains a partial implementation of the DVD Library System corresponding to our design for the *List Film* use case. The second package, called `listfilms_run`, sets up the objects of the scenario and initiates the use case.

In order to set up the scenario, objects will need to be created and initialised in quite different ways from those likely to be involved in the real system. To make this possible, various constructors and setter methods in the core system package have been declared `public` so that they are accessible from the `listfilms_run` package. In the final system (with a realistic user interface) access to these would be restricted to the core system.



Exercise 6

In NetBeans, open the project `ListFilms`, which is in the folder `M256\M256Code\Exercises\Unit8`. This implements the core classes with just the link references and methods necessary for the design of this use case (the ones identified in Exercises 4 and 5) and uses a `main()` method to set up the objects of the scenario as in Figure 3 and then initiate the use case.

Open the source code for the class `Main` in the package `listfilms_run` and study the `main()` method which is used to:

- ▶ set up the objects involved in a scenario;
- ▶ initiate the use case;
- ▶ output the results.

Thus the `main()` method takes the place of a user interface. In detail, you will find it divided into the following parts.

Part 1 – sets up the scenario as it is in Figure 3. To do this it has to instantiate some objects and set some links. No attribute values appear in Figure 3 but we have invented some for the purposes of this mock-up (otherwise the output would be rather dull!). We have chosen to use the identifiers (`film7`, `film3` and so on) as variable names.

We emphasise that none of this pretends to reflect how things would happen in the finished, fully implemented system. It is just a temporary means of creating a set of suitable objects on which the use case may be tried.

Part 2 – creates the coordinating object, `library`.

Part 3 – sends the coordinating message to `library`. In the real system this message would be sent from the user interface. However, since in this skeleton program there is no user interface, the message `getLoansAndFilms(member6)` is sent to `library` from this part of the `main()` method.

The answer to this message will be a collection holding the required objects, which ought to be the ones shown in the collection returned in Figure 8.

In the completed system this collection would be returned to the user interface; the user interface would extract the values (return dates and titles) specified in the description of the use case, and display the results as a list. Since there is no user interface here, this aspect is simulated in the next part of the method.

Part 4 – involves an iteration through the returned collection displaying, for each element, the title of the film and the return date of the loan. Again, there is absolutely no suggestion this is how it would be done in the real system, but this ‘improvised’ approach provides a way to see if the results returned are those expected.

- (a) Trace the sequence of messages that will result when the coordinating message `getLoansAndFilms(member6)` is received by the coordinating object, `library`, by examining the code for each of the methods identified in Exercise 5. The classes concerned are in the package `listfilms_core`. You may find it useful to look at the object diagram in Figure 3 to remind yourself of the starting state of the relevant part of the system. Compare these messages with the sequence of messages shown in the sequence diagram in Figure 8.

- (b) Noting that the `Loan` class initialises the issue date of each new `Loan` object to be the date on which the program is run, write down what you predict the output should be when the project is executed.

Run the project and check that the output is as expected.

Discussion.....

- (a) On receipt of the coordinating message, `library` will send the message `getLoanAndFilms()` to `member6`.
 In response to this message `member6` will send a `getFilm()` message to each of `loan23`, `loan27` and `loan29`.
 When it receives a `getFilm()` message, each `Loan` object will send a `getFilm()` message to its linked `DVD` object (`dvd2`, `dvd7` and `dvd4` respectively).
 These messages correspond to those in the sequence diagram, although the order in which the `Loan` objects are actually processed may differ.
- (b) The expected output is as follows. The order in which the films appear is unimportant.

```
Film: The African Queen
Return date: (displayed date is three days later than date on which
program is run)
Film: Spellbound
Return date: (ditto)
Film: The African Queen
Return date: (ditto)
```

In this section you have participated in the design of the *List Films* use case and investigated a working scenario for this design. In Section 5 you will study another use case, *Get Number Available*, which like *List Films* concerns returning information to the user interface. Before that, in the next two sections, you will consider the *Return DVD* and *Borrow DVD* use cases, each of which involves state changes within the core system.

3

Removing objects and links

In this section you will participate in the creation of a design for the *Return DVD* use case of the DVD Library System, which involves changing the state of the core system by removing an object and some links. (Running a working scenario for this design is left until Section 5.)

To prepare for this, you will first learn about what is generally involved in removing links (the removal of an object is relatively simple and will be discussed later on).

3.1 Removing links

What has to be done to remove a link depends upon the way in which the association is navigated. If an association is navigated in one direction only, say from P to Q , then a P object will contain a reference to the linked Q object. To remove the link all that is needed is to remove this reference. However, for a two-way link the object at each end will hold a reference to the other and, in general, both these references will need to be removed.

In *Unit 6*, Subsection 5.2, we explained that when there is a two-way association it is sensible to make the object at one end of the association responsible for managing the creation of the relevant links. To do this, the object at the 'responsible' end has, of course, to send the object at the other end a message requesting it to add a reference to the responsible object. A similar approach applies when removing a link. The object at one end will be made responsible for managing the removal and, if it is a two-way link, will enlist the help of the object at the other end by sending it a message telling it to remove the reference it holds.

In general, if an object holds a reference that needs to be removed, then that object will need to be accessed so that it can be sent a message telling it to remove the reference.

Removing a one-way link

As an example of removing a one-way link, consider the simple situation shown in the class and object diagrams in Figure 10 (a) and (b) respectively, which relate to a school system like the one you met earlier in the course.

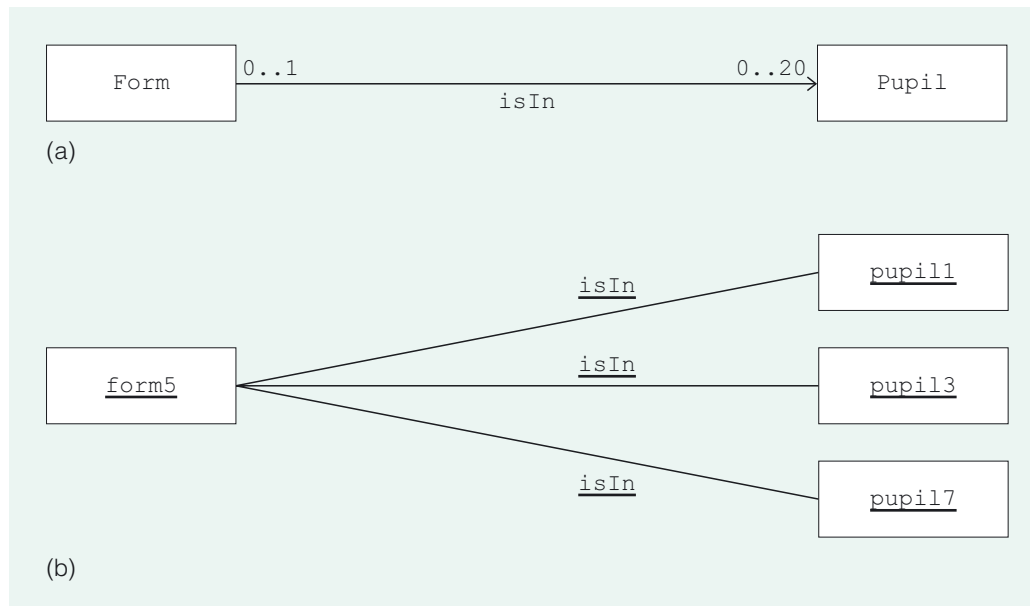


Figure 10 (a) Class diagram for part of a school system with a one-way association.
 (b) Object diagram before removal of a one-way link

Suppose that the link between `form5` and `pupil3` is to be removed: the pupil represented by `pupil3` leaves the form represented by `form5`. Since the association `isIn` is navigated only from `Form` to `Pupil` (as shown by the arrow in the class diagram), `form5` has a reference to `pupil3`, but `pupil3` does not have a reference to `form5`. `form5`'s reference to `pupil3` must be removed and so `form5` is responsible for managing the removal of the link. Of course, a message needs to be sent to `form5` to tell it to remove its reference to `pupil3` and this message will need to be sent by some object that has access to `form5`. We will assume that the sender object is called `school` and that the message is called `removePupil()`.

As you can see from the class diagram, the association `isIn` has a multiplicity greater than 1 at the `Pupil` end; this means that a `Form` object may be linked to more than one `Pupil` object (as shown in the object diagram). Thus `form5` holds (via a collection of some kind) references to *all* the linked `Pupil` objects and needs to be told which one to remove, so the message `removePupil()` needs to include the `Pupil` object, `pupil3`, as an argument. This is shown in the sequence diagram in Figure 11.

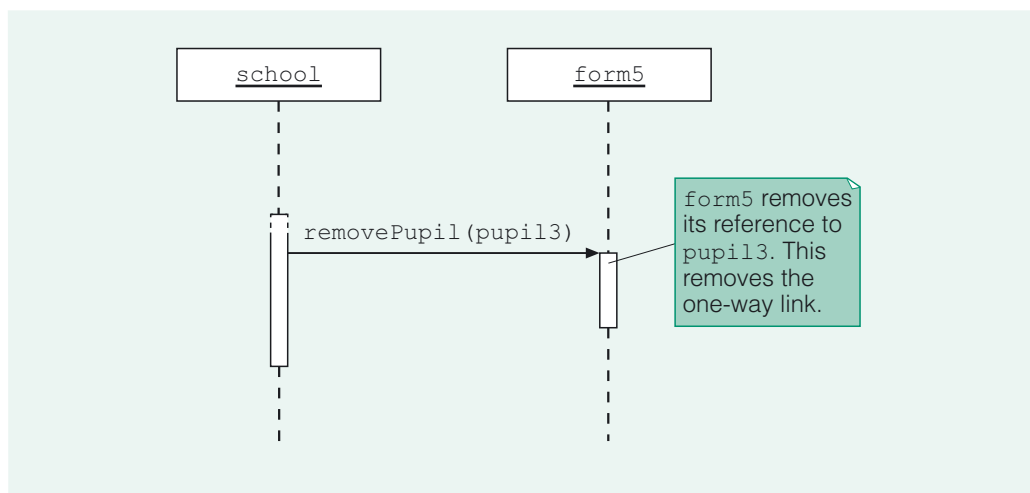


Figure 11 Sequence diagram for removal of a one-way link

Removing a two-way link

To explain what is involved in removing a two-way link, suppose now that the `isIn` association in Figure 10(a) is navigated in both directions as shown in Figure 12(a). (The object diagram in Figure 12(b) illustrating the state before the link is removed is the same as the one shown in Figure 10(b).)

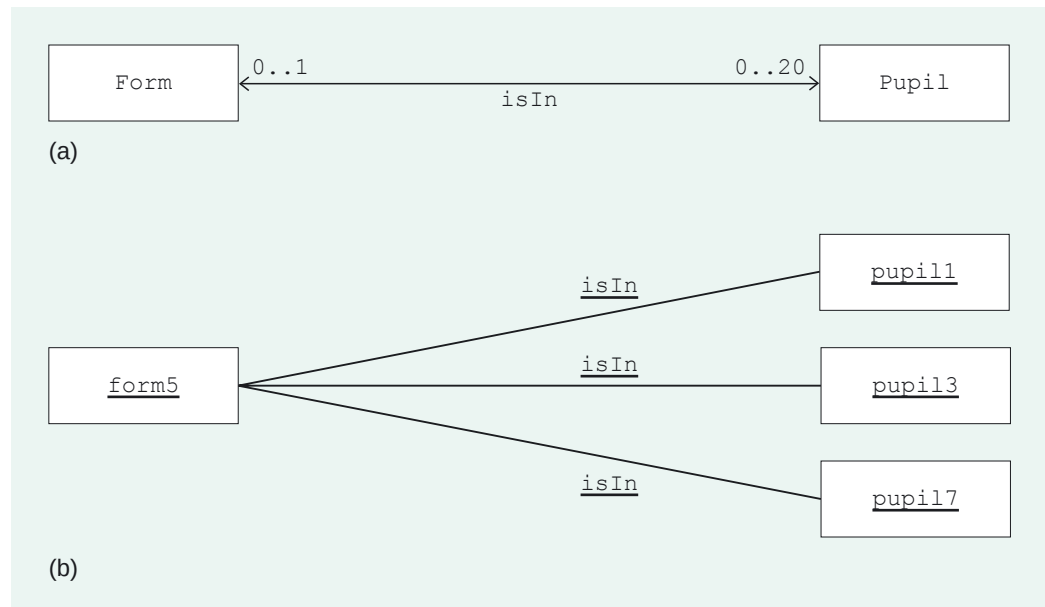


Figure 12 (a) Class diagram for part of a school system with a two-way association.
(b) Object diagram before removal of a two-way link

There is now a choice. As before, the `Form` object could be made responsible for managing the removal of the link. The only difference from the previous situation would be that, as well as removing its own reference to `pupil13`, `form5` would need to send a message to `pupil13` telling it to remove its reference to `form5`. An important point to note is that `pupil13` does not need to be told which `Form` object to remove itself from, because the multiplicity of `0..1` at the `Form` end of `isIn` means that a `Pupil` object is never linked to more than one `Form` object. The resulting sequence diagram is shown in Figure 13.

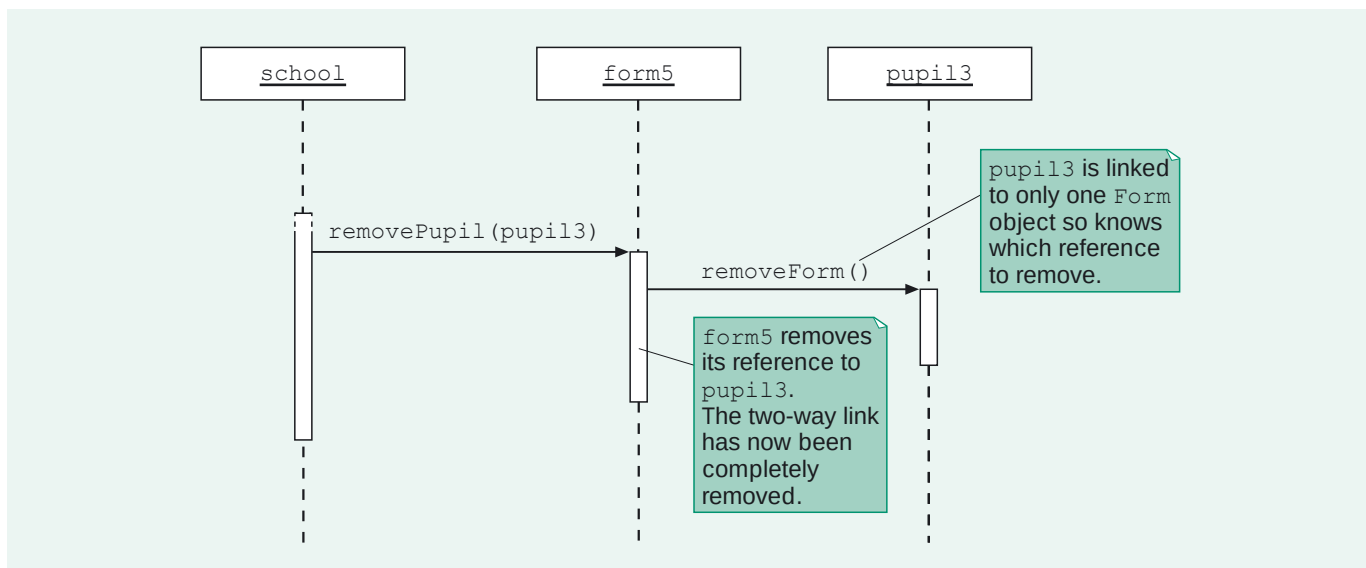


Figure 13 Sequence diagram for removal of a two-way link

Alternatively, `pupil3` could be given responsibility for managing the removal of the link. Since `pupil3` holds a reference to the single `Form` object it is linked to, it does not need to be told which `Form` object is involved, however `form5` still needs to be told which `Pupil` object has to be removed.

This approach gives the sequence diagram shown in Figure 14.

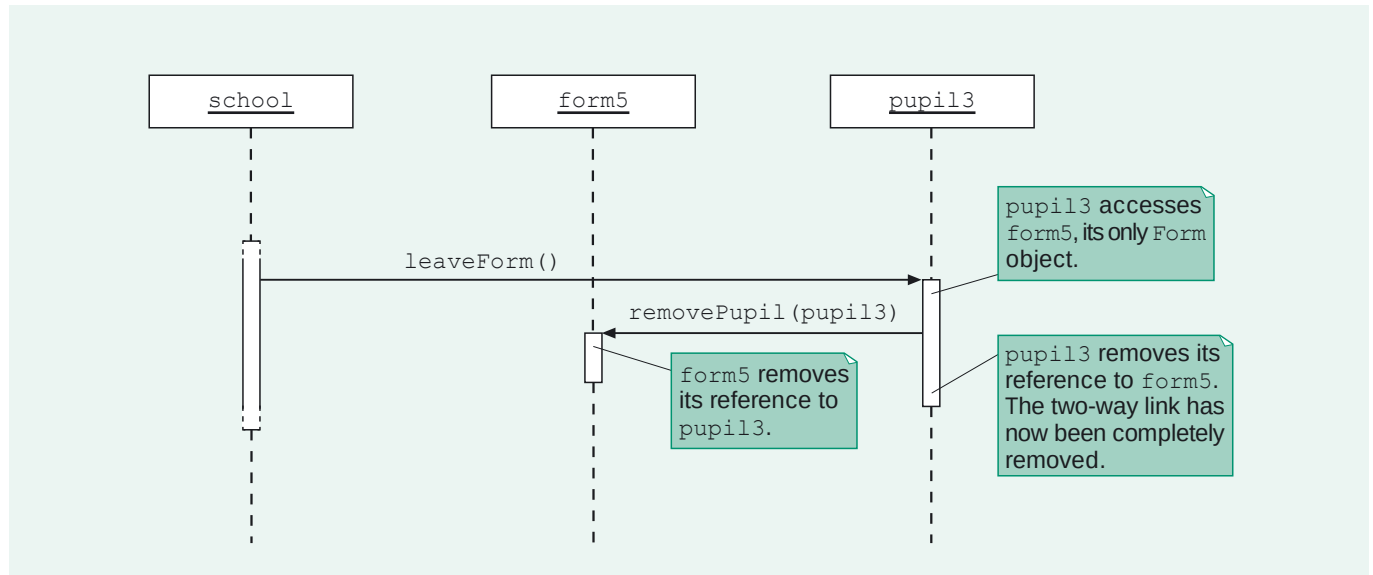


Figure 14 A different design for removing a two-way link

It is important to appreciate the part played by multiplicity. If a `Pupil` object needs to remove a reference to a `Form` object, it does not need to be told which `Form` object has to be removed as it cannot ever be linked to more than one. On the other hand if a `Form` object needs to remove a reference to a `Pupil` object it has to be told which one as it may be linked to many `Pupil` objects.

3.2 The *Return DVD* use case

Having set the scene by discussing what is involved in removing links in general, we now turn to the use case that is the subject of the remainder of this section – *Return DVD* – whose design, as you will see, involves removing links. As a reminder, here is a description of the use case.

The staff member identifies the DVD. The system records the fact that this copy of the film is now available for loan and that it is no longer on loan to the member who returned it.

Stage 1 – choose the objects for a typical scenario

When analysing use cases that involve changing the state of a system, before and after states must be considered. This involves drawing an object diagram showing some typical objects *before* the use case and then another object diagram to show the situation *after* the use case has been carried out. This will enable you to identify what actions are needed in this particular scenario.

In Figure 14 the arrow for the message `removePupil(pupil3)` from `pupil3` to `form5` goes from right to left. This is perfectly acceptable; although the usual arrangement is for messages to flow from left to right, there are occasions where right to left message arrows are used e.g. to avoid changing the order of the objects.

Exercise 7

- (a) Figure 15 shows a 'before' state for the *Return DVD* use case. Sketch the corresponding state *after* dvd6 has been returned.

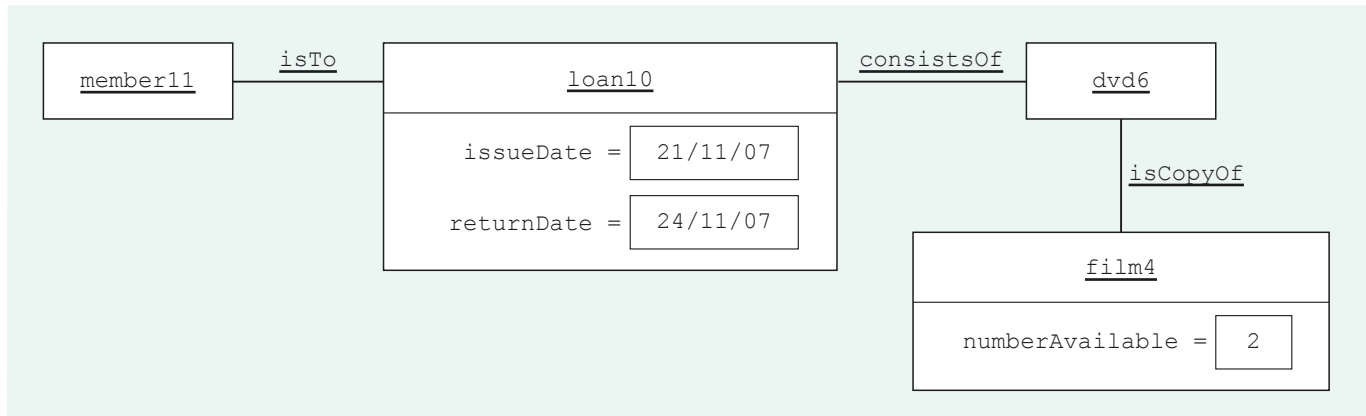


Figure 15 Object diagram for the scenario before *Return DVD*

- (b) There is a pre-condition that must be true before a DVD can be returned. Write it down in your own words.

Discussion.....

- (a) Figure 16 shows that after the use case completes, all the links involving loan10 have been removed and loan10 itself has been removed (we will discuss what this means shortly). Note that the numberAvailable attribute of film4 has been incremented.

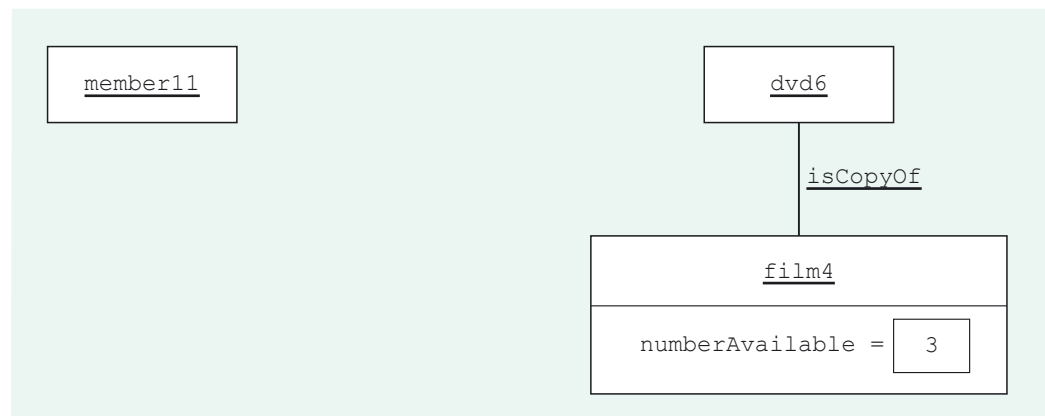


Figure 16 Object diagram for the scenario after *Return DVD*

- (b) Although not explicitly mentioned in the use case description it is obviously not possible to return a DVD unless it is already on loan. So there is a pre-condition that the DVD must currently be on loan.

You may also have considered the statement in the requirements that 'A borrowed DVD must be returned within three days.' This could be interpreted as meaning that a DVD cannot be returned if the current date is after the returnDate for the loan. For the purposes of this exercise we will assume it is intended that some sort of fine should be imposed for a late return; this would perhaps mean alternative behaviour rather than the imposition of another pre-condition for *Return DVD*. In the requirements for this simple system this complication has, for simplicity, been ignored so the *Return DVD* use case need not do anything different if a DVD is returned late.

The design for *Return DVD* will be based on a typical, valid scenario, whose objects are illustrated in Figures 15 and 16, in which the pre-condition is met.

Stage 2 – draw a sequence diagram

SAQ 4

Draw a sequence diagram showing just the communication between the user interface and the coordinating object for the *Return DVD* scenario.

ANSWER.....

Our diagram is shown in Figure 17. You may have used a different name for the message but the argument should be `dvd6` to correspond with the scenario we have chosen in Figures 15 and 16. Note that there is no answer returned to the user interface.

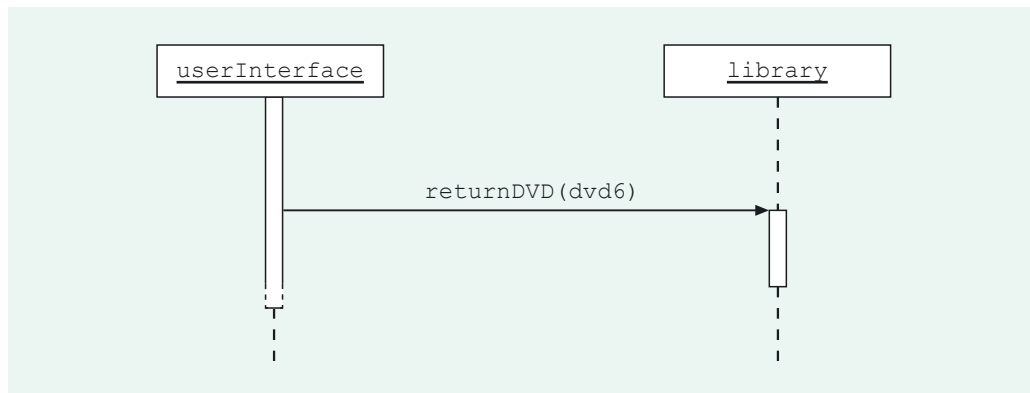


Figure 17 Communication between the user interface and the coordinating object for *Return DVD*

Stage 3 – specify the coordinating method

Next, having considering what is required for the particular scenario, the coordinating method can be specified as follows.

```
void returnDVD(DVD aDVD)
```

Pre-condition: `aDVD` must be linked to a `Loan` object.

Post-condition: the `Loan` object linked to `aDVD` and all links involving it are removed. `numberAvailable` of the `Film` object linked to `aDVD` is incremented.

Stage 4 – devise a walk-through and extend the sequence diagram

SAQ 5

When the *Return DVD* use case is carried out in our scenario, the coordinating message, `returnDVD(dvd6)`, is sent from the user interface to the coordinating object, `library`. On receipt of this message what is the first thing `library` needs to ensure is done?

ANSWER.....

Since in our scenario the pre-condition is satisfied, the first thing `library` needs to ensure is that the `Loan` object, `loan10`, linked to `dvd6` is accessed, so that `loan10` and its links with `dvd6` and `member11` can be removed. (Recall that in this unit we are only concerned with the first design iteration. Steps to ensure that no invariants are violated would be added in a later iteration.)

Thus the walk-through for our scenario starts as follows.

- Given: the DVD object, dvd6.
- 1 Access the Loan object linked to dvd6, which in this case is loan10.

SAQ 6

Refer to the object diagram in Figure 15. Which association needs to be navigated, and in which direction, in order to implement step 1 in the above walk-through?

ANSWER.....
The association `consistsOf` needs to be navigated from DVD to Loan.

Having accessed the relevant Loan object, the system needs to remove the links to this Loan object. The approach to link removal will be based upon the discussion contained in Subsection 3.1, although, as you will see, some of the aspects of this use case differ from the school example considered there.

The navigations are currently as shown in Figure 18. The `consistsOf` association is now a two-way association since, for step 1 of the walk-through for *Return DVD*, it needs to be navigated from DVD to Loan, as well as (for the *List Films* use case) from Loan to DVD.

Since Figure 18 is only concerned with navigations of associations, the coordinating class, which at present has no associations, is not shown.

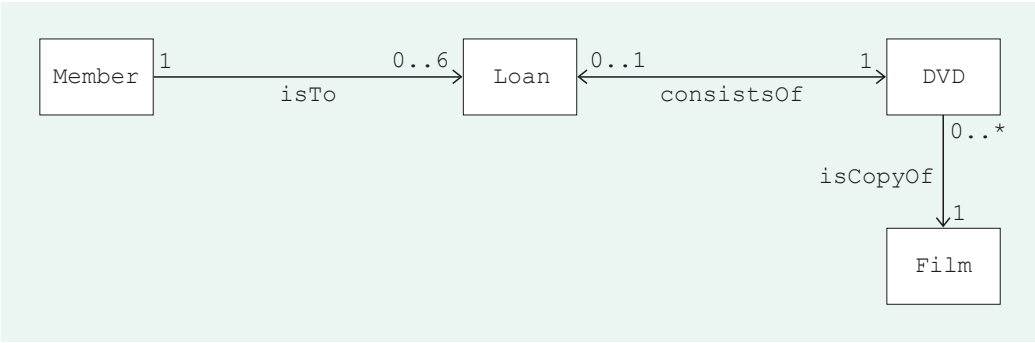


Figure 18 The navigations so far

SAQ 7

What needs to be done to remove the `isTo` link? (Refer to the object diagram in Figure 15.)

ANSWER.....
The reference to loan10 held by member11 must be removed.

To see how to remove the reference to loan10 that is held by member11 remember that the only information supplied to the coordinating object is the DVD object, dvd6. This means that member11 needs to be accessed from dvd6, via loan10, in order for it to be sent a message telling it to remove its reference to loan10.

This leads to the apparently counter-intuitive result that, to remove the link which supports navigation in one direction, there needs to be navigation in the other direction! The class diagram of Figure 19 records that `isTo` must now be a two-way association.

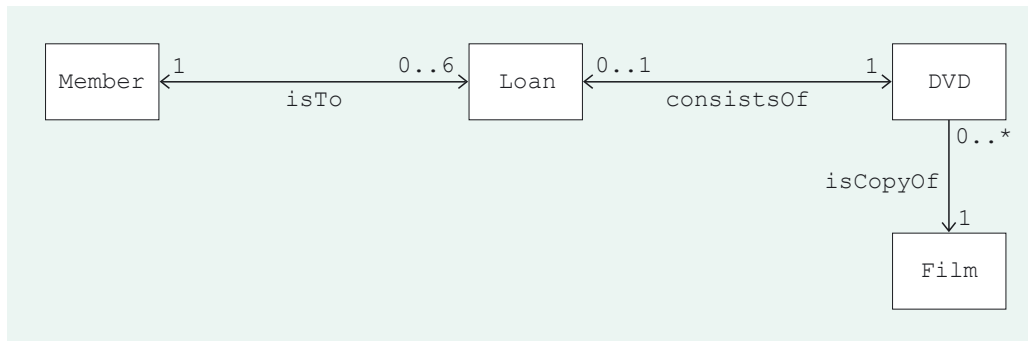


Figure 19 The navigation of the `isTo` association

`dvd6`'s reference to `loan10` will also need to be removed, but this is straightforward since `dvd6` is provided to the coordinating object.

Once both `member11` and `dvd6` have removed their references to `loan10`, there will no longer be any reference to `loan10` in the core system.

In Java, the process of garbage collection disposes automatically of any object that is not referenced from somewhere in the system (and which is therefore unusable), by reclaiming its memory space. Since our target language is Java, our designs will assume that when all references to an object have been destroyed, the object will then be automatically destroyed as well, so no explicit design step is needed to remove redundant objects.

In other languages, an explicit programming step may be necessary to dispose of an object in this way.

Hence `loan10` will be destroyed once both `member11` and `dvd6` have removed their references to it. There will thus be no need for `loan10` to remove its references to `member11` or `dvd6`. (See Figure 20.)

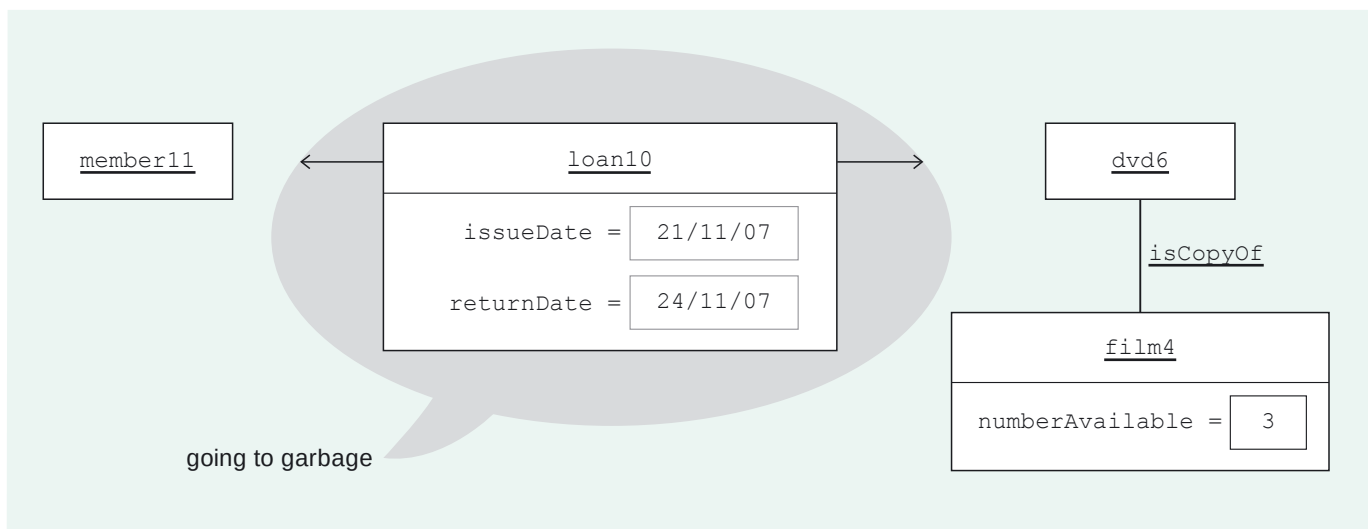


Figure 20 `loan10` is about to be garbage collected as the references in `member11` and `dvd6` have been removed (`loan10` still retains its references to `member11` and `dvd6`)

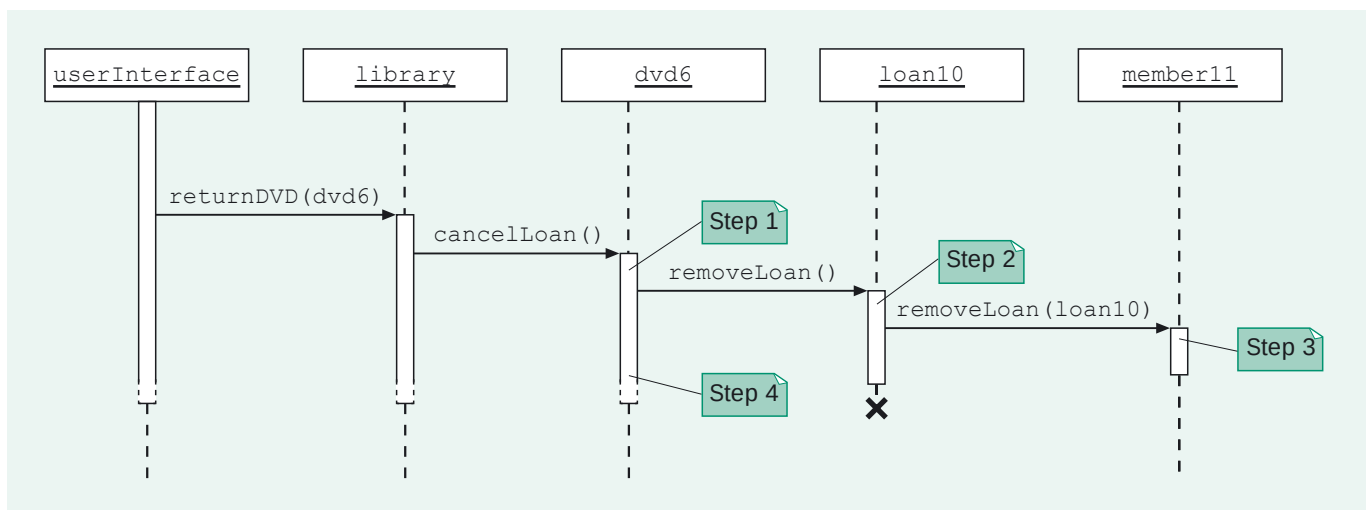
In this respect, the situation is different from the general case considered in the school example. Although both links (i.e. the link of the `isTo` association and the link of the `consistsOf` association) are two-way, only the reference at one end of each link needs to be removed. Thus there is no need to consider which object has responsibility for managing the removal of each link – the object at the relevant end will simply remove its reference.

The walk-through is now as follows. It will be refined further, as we discuss shortly.

Given: the DVD object, `dvd6`.

- 1 Access the `Loan` object linked to `dvd6`, which in this case is `loan10`.
- 2 Access the `Member` object linked to `loan10`, which in this case is `member11`.
- 3 `member11` removes its reference to `loan10`.
- 4 `dvd6` removes its reference to `loan10`.

The sequence diagram in Figure 21 illustrates one way of achieving these walk-through steps. In this design we might describe `dvd6` as having overall responsibility for managing the removal process – it initiates the removal of the `Loan` object and all the relevant links. Notice the use of **X** in the sequence diagram to indicate the destruction of an object.



Note that in Figure 21 the activation rectangles for `library` and `dvd6` have not been completed; this is because the sequence diagram is to be refined further.

Figure 21 Removing the two links to `loan10`

SAQ 8

Explain why in Figure 21 the `removeLoan()` message sent to `member11` requires an argument.

ANSWER.....

The `isTo` association has a multiplicity of 0..6 at the `Loan` end, so the `Member` object needs to be told which `Loan` reference to remove.

In step 4 of the above walk-through, the purpose of which is to remove the `consistsOf` link, an important point has yet to be addressed.

Exercise 8

In addition to removing the reference, what else needs to be done as part of step 4 when the `consistsOf` link is removed? (Hint: look at the 'before' and 'after' diagrams in Figures 15 and 16.)

Discussion.....

As you can see from the 'before' and 'after' diagrams, the `numberAvailable` attribute of `film4` has to be incremented. This is because invariant 2 of the DVD Library System initial structural model requires that:

For any `Film` object, the value of its `numberAvailable` attribute is equal to the number of `DVD` objects it is linked to which are not linked to a `Loan` object.

In order to increment the `numberAvailable` attribute of `film4`, `film4` must (as part of the process of disconnecting `dvd6` from `loan10`) be accessed and sent a message telling it to increment its `numberAvailable` attribute. This message can be sent by `dvd6` after it has removed its reference to `loan10`.

Putting all this together, we get the following walk-through for our scenario.

Given: the DVD object, `dvd6`.

- 1 Access the `Loan` object linked to `dvd6`, which in this case is `loan10`.
- 2 Access the `Member` object linked to `loan10`, which in this case is `member11`.
- 3 `member11` removes its reference to `loan10`.
- 4a `dvd6` removes its reference to `loan10`.
- 4b Access the `Film` object linked to `dvd6` which in this case is `film4`.
- 4c Increment the `numberAvailable` attribute of `film4`.

Exercise 9

Draw a complete sequence diagram corresponding to the above walk-through.

Discussion.....

Here is our sequence diagram (Figure 22). You may have used a different name for the `incrementNumberAvailable()` message but so long as the name gives an indication of what the message does this is not important. We have included notes to indicate the steps of the walk-through, but that is not essential.

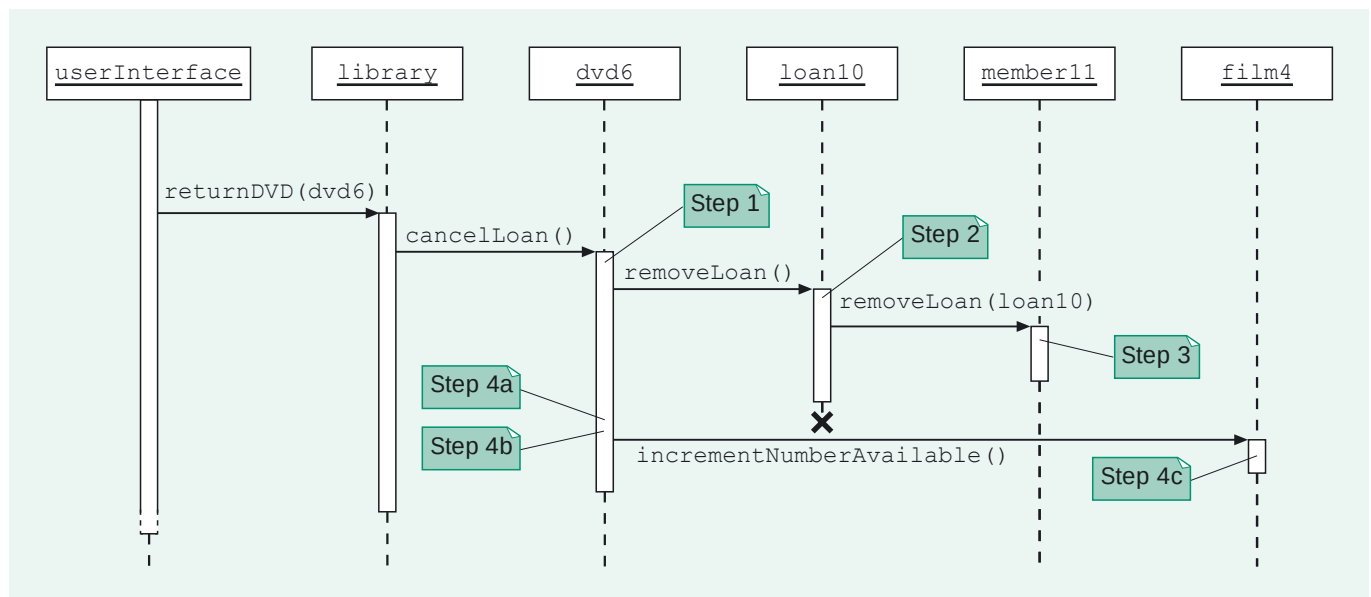


Figure 22 Sequence diagram for *Return DVD*

Exercise 10

There is an alternative design for achieving step 3 that involves `dvd6` sending the `removeLoan(loan10)` message to `member11`. This uses a fork instead of a cascade. Draw a sequence diagram to show how this can be achieved. Most of this sequence diagram should be the same as Figure 22: only the parts relating to steps 2 and 3 need changing.

Discussion.....

For `dvd6` to send a message to `member11`, it first needs to get a reference to `member11`. It can do this by sending a `getMember()` message to `loan10`. The required sequence diagram is shown in Figure 23.

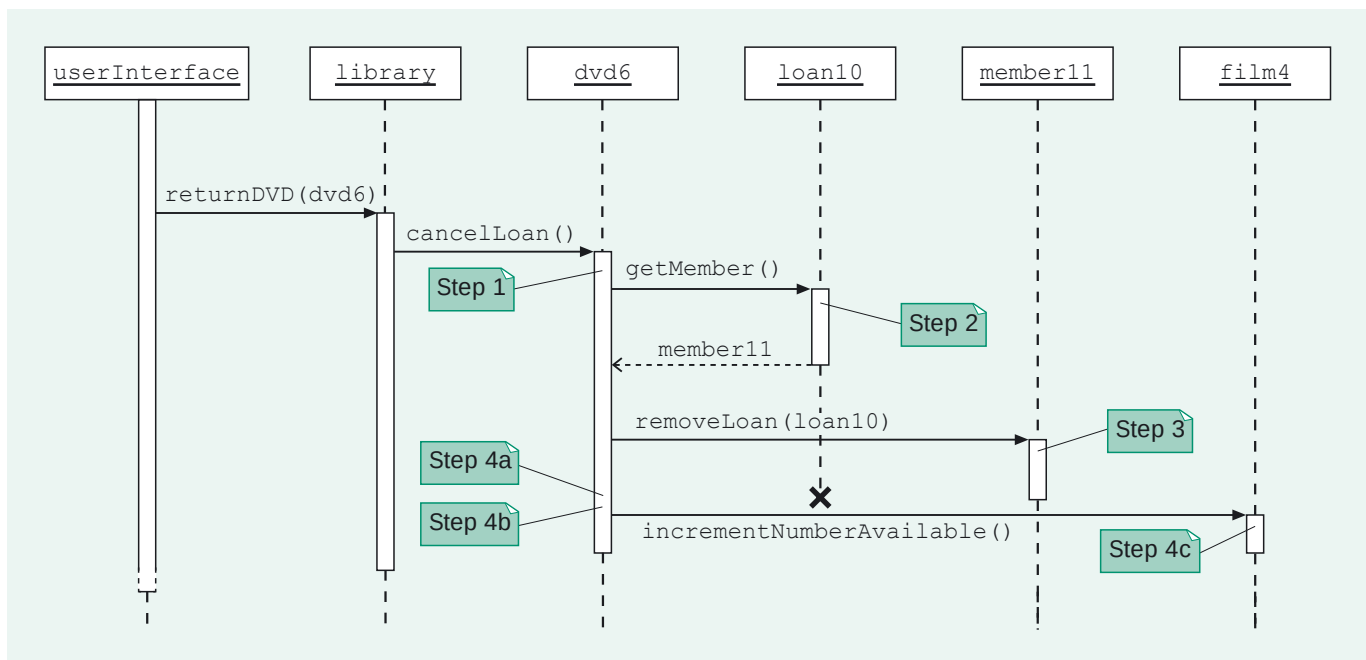


Figure 23 `dvd6` tells `member11` to remove its reference to `loan10`

For the purposes of this unit we will use the design for *Return DVD* shown in Figure 22, preferring it to the alternative design involving a `DVD` object sending a message to a `Member` object to which it is not linked, which would tend to increase coupling.

Once these design decisions have been made, the final stage is to determine the consequences for the structural model.

Stage 5 – amend the structural model

The navigations needed have already been indicated, and there will be corresponding responsibilities for holding link references to be added to the class descriptions. Furthermore, as well as the coordinating method, there will be additional methods corresponding to the messages in the sequence diagram.

Consider the `DVD` class first. To determine what methods are needed in the protocol of the `DVD` class, examine the sequence diagram in Figure 22 to identify the messages that are sent to the `DVD` objects. In this case there is only one `DVD` object, `dvd6`, and it is sent only one message, `cancelLoan()`. This means that a corresponding method is needed in the protocol of the `DVD` class. As the message does not show an answer being returned the return type of the method is `void`.

The specification of the method is therefore:

```
void cancelLoan()
```

Post-condition: all references to the linked `Loan` object from the receiver and from all other objects are removed. `numberAvailable` of the linked `Film` object is incremented.

SAQ 9

Give the specifications of the methods that should be added to the protocols of the `Member`, `Loan` and `Film` classes on the basis of the above design.

ANSWER.....

In the `Member` class:

```
void removeLoan(Loan aLoan)
```

Post-condition: the reference to `aLoan` is removed.

In the `Loan` class:

```
void removeLoan()
```

Post-condition: the reference to the receiver held by the `Member` object linked to the receiver is removed.

In the `Film` class:

```
void incrementNumberAvailable()
```

Post-condition: `numberAvailable` is incremented.

In this section you have learnt what is generally involved in removing links and you have used this knowledge to participate in the creation of a design for the *Return DVD* use case. The practical work involving a working scenario for *Return DVD* is postponed until Section 5, after we have investigated the remaining use cases. Although it is possible to design an implementation for the *Return DVD* use case first, it cannot actually be tried out until there are some borrowed DVDs to return!

4

Creating objects and links

In the previous section, you studied the design of a use case, *Return DVD*, which changes the core system by *removing* an object and links. In this section, you will investigate the design of another use case, *Borrow DVD*, that changes the core system, but this time by *creating* an object and links. Recall that *Unit 6*, Subsection 4.2, introduced you to what is generally involved in creating objects and links.

4.1 The *Borrow DVD* use case

Here is a description of the *Borrow DVD* use case.

The staff member identifies the DVD and the member who is borrowing it. The system records the fact that this copy of the film is no longer available for loan, that it is on loan to the member identified, and the date on which it was borrowed.

Stage 1 – choose the objects for a typical scenario

For *Borrow DVD* we shall use the same objects and links as in the scenario for *Return DVD* except that, of course, the 'before' and 'after' states will be interchanged as shown in Figures 24 and 25.

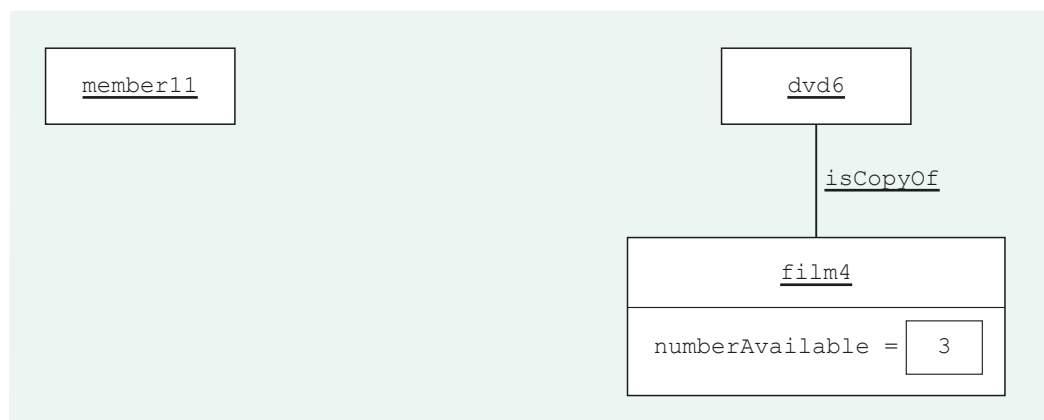


Figure 24 Object diagram for the scenario before *Borrow DVD*

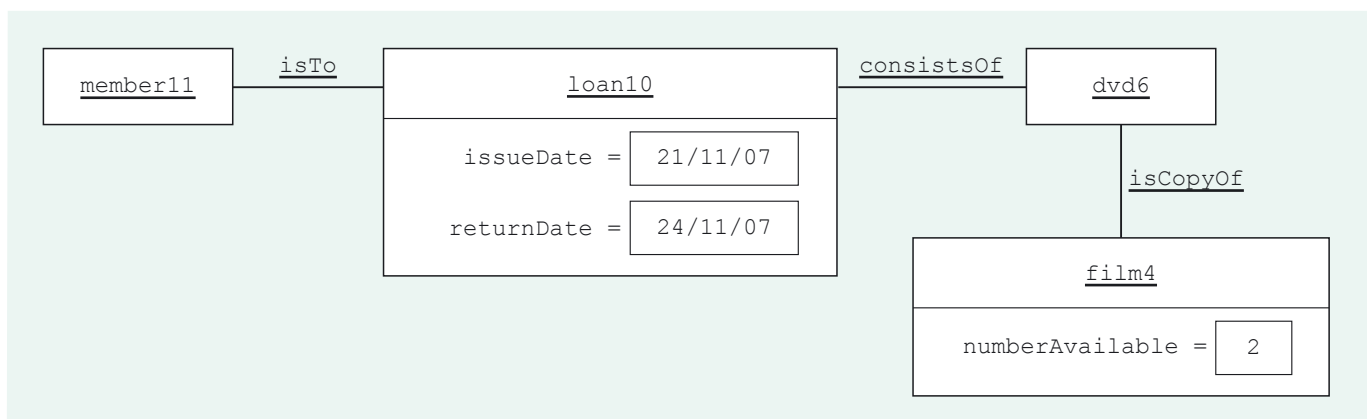


Figure 25 Object diagram for the scenario after *Borrow DVD*

Stage 2 – draw a sequence diagram

Having decided on the objects for an example scenario, a sequence diagram showing the communication between the user interface and the core system can be sketched. We will call the coordinating message `borrowDVD()`.

SAQ 10

What arguments should the `borrowDVD()` message have?

ANSWER.....

It needs two arguments, a DVD object and a Member object: in this scenario, `dvd6` and `member11`. This follows from the first sentence in the use case description: 'The staff member identifies the DVD and the member who is borrowing it.'

The sequence diagram is in Figure 26.



Figure 26 Communication between the user interface and the coordinating object for *Borrow DVD*

Stage 3 – specify the coordinating method

Exercise 11

There are two pre-conditions that must be true before a member will be allowed to borrow a DVD. Write down what these are. (You may like to reread the section 'System domain' in the DVD Library System requirements document in Subsection 2.1.)

Discussion.....

The requirements document says:

A library member can borrow any DVD of a film that is not currently on loan.
However, no member can have more than six DVDs on loan at once.

This gives two pre-conditions.

The DVD must not currently be on loan.

The member must have fewer than six DVDs on loan at present.

The design for *Borrow DVD* will be based on a typical, valid, scenario in which it is assumed that both these pre-conditions are met. Consideration of situations in which the pre-conditions are not met would be left to a later iteration of the design process (although M256 will not actually consider this later part of the process for the DVD Library System).

Having identified the pre-conditions, the specification of the coordinating method can be drawn up. From the object diagrams it can be seen that the coordinating method needs to get the system to create a new `Loan` object, with the correct attribute values and linked to the given `Member` and `DVD` objects, as well as getting the `numberAvailable` attribute of the `Film` object decremented.

```
void borrowDVD(DVD aDVD, Member aMember)
```

Pre-condition: `aDVD` is not linked to a `Loan` object and `aMember` is linked to fewer than six `Loan` objects.

Post-condition: a new `Loan` object `aLoan` is created with the `issueDate` and the `returnDate` set. `aLoan` is linked to `aDVD` and to `aMember`. `numberAvailable` of the `Film` object linked to `aDVD` is decremented.

Stage 4 – devise a walk-through and extend the sequence diagram

The design for this use case will, as you have seen, involve creating a `Loan` object and creating links involving this object. We first discuss what this entails.

Creating the new `Loan` object

Consider first how the creation of a `Patient` object was dealt with in *Unit 6*, Subsection 5.2, as part of the *Admit Patient* use case for the Hospital System. The relevant part of the sequence diagram for this use case is shown in Figure 27.

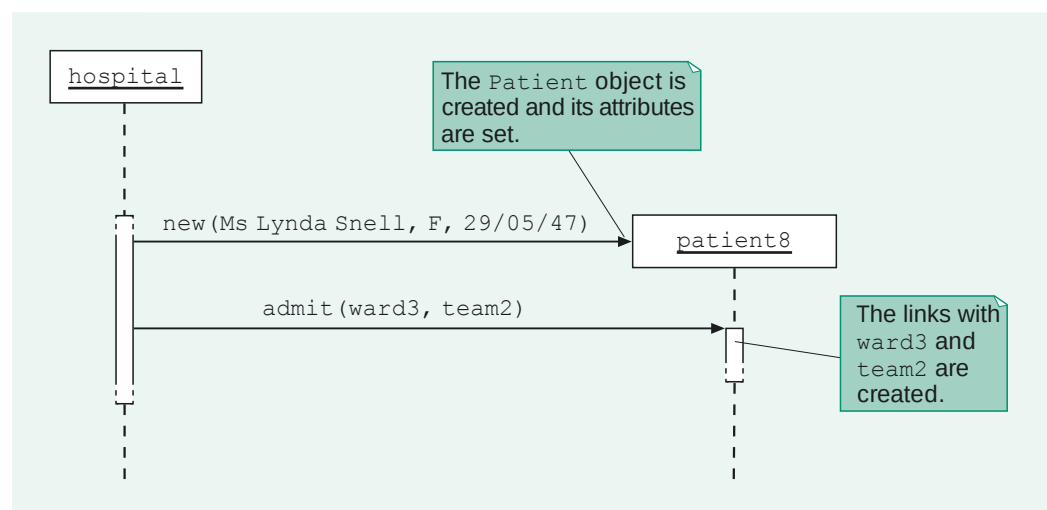


Figure 52 in *Unit 6* gives the complete sequence diagram.

Figure 27 Part of the sequence diagram for *Admit Patient* showing the creation of a new `Patient` object and its links

In this design, the attributes (`name`, `sex`, etc.) are set when the `Patient` object is created but the links are set separately when the `admit(ward3, team2)` message is sent. Informally it could be said that the patient is first created and then admitted to the hospital.

Now the same design *could* be used when creating a new `Loan` object. In other words first create the `Loan` object with its dates set and then use a separate message to link the `Loan` object to the `DVD` and `Member` objects.

In this case, however, unlike the situation with `Patient`, the `Loan` object exists solely to represent the event of a particular member borrowing a particular DVD on a certain date. A `Loan` object simply represents a connection between a particular `DVD` object and a particular `Member` object. It has no other purpose and during its lifetime the `Loan` object is never required to be linked to a different `DVD` object or to different `Member` objects other than the ones it began with. It would be positively dangerous to provide

Compare this with the case of a `Patient` object, which might be linked to a different `Ward` (if the patient transferred wards for example).

Loan with any methods that set the links, in case an error led to the links accidentally getting altered and becoming incorrect.

Thus there is a choice of design, but the above discussion indicates that, rather than giving methods to Loan for creating links, it would be better to make a Loan object set its links at the time of its creation, by providing Loan with a constructor

`Loan(aDVD, aMember)` and including the code that sets the links in the body of this constructor. This approach is illustrated in the sequence diagram in Figure 28.

Do not be concerned if you have not previously encountered a constructor like this – you will see in Exercise 15 how this is achieved in code.

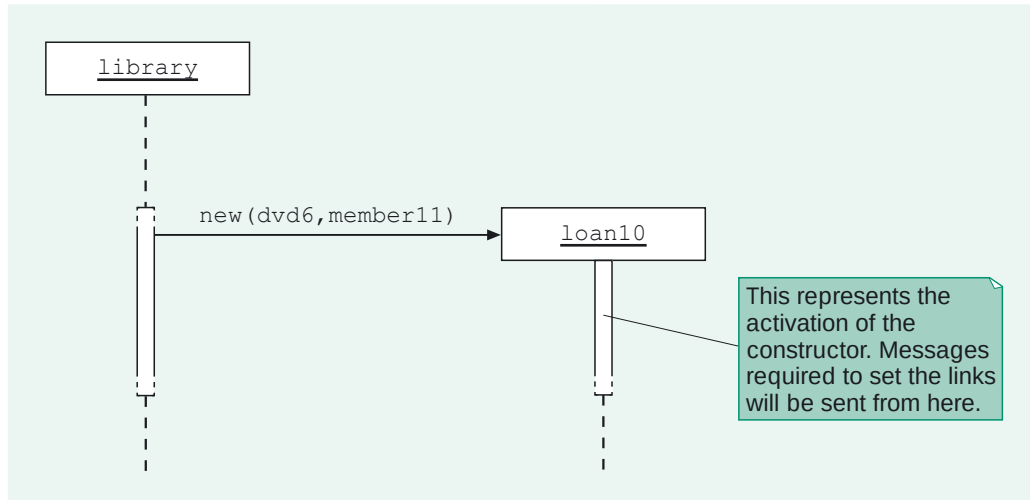


Figure 28 Creating a new Loan object and setting its links as a single 'operation'

Notice that the activation rectangle comes straight out of the loan10 object; it does not represent an activation in response to a message, but is part of the 'construction' of the new object.

Managing the link creation

This use case must result in two two-way links – `consistsOf` and `isTo` – being created. Whenever there is a two-way link, it is vital to ensure consistency. If a reference is set in one direction, it must be guaranteed that the reference going the other way is always set as well.

As has been pointed out several times, the standard way to do this is to make the object at one end responsible for ensuring that both references are set – that is, for managing the process of link creation.

In this case both of the two-way links will be set from within a constructor rather than a method. This means that there is no choice of which object will manage the link creation – for each link, it is the Loan object that is going to be responsible for managing the process. The Loan constructor will initiate this.

So, in the design loan10 will be made responsible for sending messages to dvd6 and member11, instructing them to record a reference to loan10. loan10 will, of course, also set its own links to these objects.

Now the principles have been considered, let us pull it all together.

Another design principle is at play here: if a set of tasks should always be carried out together, give one object overall responsibility for them.

The walk-through and sequence diagram

The following outline walk-through and sequence diagram for the scenario contains general steps indicating link creation but no details of how this is to be done – they will shortly be refined to include these details.

Given: the DVD object, dvd6, and the Member object, member11.

- 1 Create a new Loan object, loan10, with `issueDate` set to the current date and `returnDate` set to the date three days later.
- 2 Create a `consistsOf` link between loan10 and dvd6.
- 3 Create an `isTo` link between loan10 and member11.

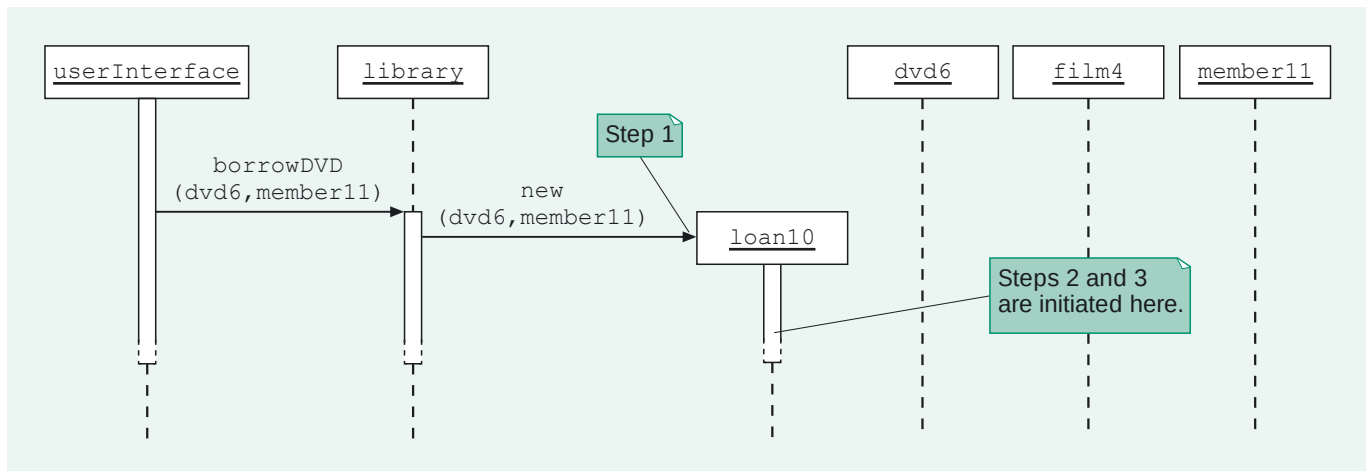


Figure 29 An outline sequence diagram for *Borrow DVD*

To complete the walk-through and sequence diagram it is necessary to refine steps 2 and 3. As discussed above, both the links are two-way links and so messages will need to be sent to member11 and dvd6 to tell them to record references to loan10. However for step 3 there is an additional substep.

Exercise 12

Consider the following invariant (invariant 2) for the DVD Library System.

For any `Film` object, the value of its `numberAvailable` attribute is equal to the number of `DVD` objects it is linked to which are not linked to a `Loan` object.

If this invariant is to be preserved, what needs to be done as part of step 2 in the above walk-through when dvd6 is linked to loan10?

Discussion.....

The value of the `numberAvailable` attribute of film4 will need to be reduced by 1.

Taking the above points into account you should now be able to complete the walk-through and sequence diagram.

Exercise 13

Complete the following walk-through for the scenario for *Borrow DVD* whose objects are illustrated in Figures 24 and 25.

Given: the DVD object, dvd6, and the Member object, member11.

- 1 Create a new Loan object, loan10, with `issueDate` set to the current date and `returnDate` set to the date three days later.
- 2a loan10 records a reference to ____.
- 2b ____.

- 2c Access the ____ object linked to ____ which in this case is ____.
- 2d Decrement the ____.
- 3a `loan10` records a reference to ____.
- 3b ____.

Discussion.....

Given: the DVD object, `dvd6`, and the Member object, `member11`.

- 1 Create a new Loan object, `loan10`, with `issueDate` set to the current date and `returnDate` set to the date three days later.
- 2a `loan10` records a reference to `dvd6`.
- 2b `dvd6` records a reference to `loan10`.
- 2c Access the `Film` object linked to `dvd6` which in this case is `film4`.
- 2d Decrement the `numberAvailable` attribute of `film4`.
- 3a `loan10` records a reference to `member11`.
- 3b `member11` records a reference to `loan10`.

Exercise 14

Draw a sequence diagram corresponding to the walk-through given in the answer to Exercise 13, labelling it with the step numbers.

Discussion

Our sequence diagram is given below.

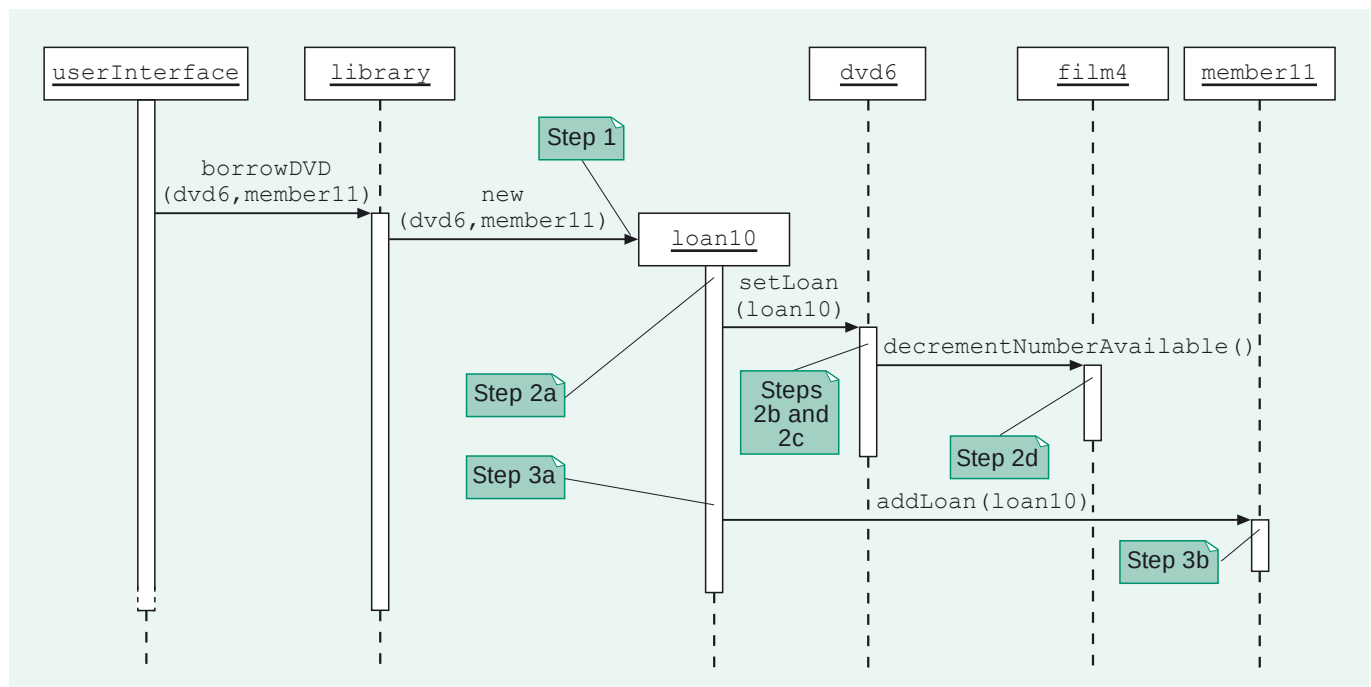


Figure 30 Sequence diagram for *Borrow DVD*

You may have used different names for some of the messages; this is fine. Note that we have called the message sent to `dvd6` `setLoan` but the one sent to `member11` we called `addLoan`. Though details of message names are not crucial at this stage, our choice reflects the fact that a member can have more than one loan, so `loan10` is added to the collection of any `Loan` objects that `member11` already has. On the other hand a DVD can be subject to, at most, one loan at a time so, for `dvd6`, its `Loan` object is set rather than

another one added. Notice also that both `dvd6` and `member11` need to be provided with `loan10` as an argument otherwise they will not know which `Loan` object to link to. Compare this with the *Return DVD* situation where `dvd6` was able to work out for itself what `Loan` object reference needed to be removed.

Stage 5 – amend the structural model

The consequences for the structural model of this design for *Borrow DVD* are simply the addition of a constructor and of some methods to the protocols of the relevant classes; the only access step uses a navigation (of `isCopyOf` from `DVD` to `Film`) that is already recorded in our structural model. The next SAQ simply asks you to identify these methods.

SAQ 11

For each of the classes `DVD`, `Member` and `Film` give the return type, name and argument(s) for any methods required for the above design for *Borrow DVD*.

ANSWER.....

In `DVD`:

```
void setLoan(Loan aLoan)
```

In `Member`:

```
void addLoan(Loan aLoan)
```

In `Film`:

```
void decrementNumberAvailable()
```

In addition to the methods listed in the answer to SAQ 11, the design for *Borrow DVD* requires the specification of a constructor for the class `Loan`, as follows:

```
Loan(DVD aDVD, Member aMember)
```

Post-condition: initialises a new `Loan` object `aLoan`. `aLoan` records references to `aDVD` and `aMember`. `aDVD` and `aMember` record references to `aLoan`. The `numberAvailable` attribute of the `Film` object linked to `aDVD` is decremented.

In this section, you have participated in the creation of a design for the *Borrow DVD* use case which changes the state of the core system by creating an object and links. In the next section you will consider the *Get Number Available* use case, which will necessitate consideration of derived attributes, and you will explore the remaining working scenarios.

5

Derived attributes and the *Get Number Available* use case

In Sections 3 and 4 designs were created for the *Return DVD* and *Borrow DVD* use cases of the DVD Library System. In this section you will:

- ▶ consider the design of the *Get Number Available* use case;
- ▶ explore working scenarios of the *Borrow DVD* and *Return DVD* designs.

The *Get Number Available* use case has some interesting features, although, as you will see, what has to be done is actually very simple. One feature of interest is that the use case involves obtaining the value of a derived attribute, so this section will start with a discussion of some issues involved in implementing, or not implementing, derived attributes.

5.1 Derived attributes

The DVD Library System has the following invariants.

- 1 For each `Loan` object, the difference between the values of its `issueDate` and `returnDate` attributes is 3 days.
- 2 For any `Film` object, the value of its `numberAvailable` attribute is equal to the number of `DVD` objects it is linked to which are not linked to a `Loan` object.

The initial structural model also indicates (by preceding their names with a /) that there are two derived attributes: `returnDate` in the class `Loan` and `numberAvailable` in the class `Film`. This means that the values of these attributes can be calculated from other information in the system.

Consider first the attribute `returnDate`.

The invariants are listed in the initial structural model for the DVD Library System in Section 9 of the *Handbook*.

SAQ 12

If `returnDate` is not implemented as an instance variable, how can the system calculate a `returnDate` attribute value?

ANSWER.....

The system can calculate the value of `returnDate` from the value of `issueDate` by using invariant 1; that is, by adding 3 to the value of `issueDate`.

Of course the system could equally well use invariant 1 to calculate an `issueDate` value from a `returnDate` value. There seems little point therefore in implementing both these attributes as instance variables, but it needs to be decided which to implement. Consideration of the use that is made of each attribute in the use cases can help in this decision, though there is actually very little to consider here.

The *List Films* use case requires the return dates to be displayed. The *Borrow DVD* use case mentions recording the issue date but it does not actually *use* the issue date. Indeed, it would appear that there is no need for the issue date anywhere other than as a step in calculating the return date. It would therefore seem more appropriate to implement `returnDate` rather than `issueDate`.

On the basis of the above discussion we will implement `returnDate`, but not `issueDate` – this decision does not affect the designs that have so far been created.

The values of the `numberAvailable` attribute of `Film` can also be calculated by the system, in this case by using invariant 2, so we also need to decide whether or not to implement `numberAvailable` as an instance variable. Before doing that we investigate the *Get Number Available* use case to determine what it involves.

5.2 The *Get Number Available* use case

Here is the description of this use case.

The staff member identifies the film. The system displays the number of copies available.

In order for the member of staff to identify the film, the user interface must already have a reference to the `Film` object. It follows that the core system does not need to do anything other than provide the user interface with a collection of all the `Film` objects to allow the user to choose the relevant one. For the user interface to get the required information from the selected `Film` object it only needs to send the appropriate getter message to that object. From all of this, three things follow.

- 1 `Film` must have a public getter method that returns the value of the attribute `numberAvailable` – call this method `getNumberAvailable()`.
- 2 `LibCoord` (the coordinating class) must have a public method that returns a collection of all the `Film` objects – call this method `getFilms()`.
- 3 This use case does not require any coordinating method; once the required `Film` object has been selected, the user interface has a reference to the object from which it can obtain the information required, so no further contribution from the coordinating object is needed.

Regarding the first point, recall that in *Unit 7* it was decided that we would have a public getter method for each attribute of each object in the core system. By doing this the user interface can display whichever attribute values of an object are required. Following this convention would give `Film` a method `getNumberAvailable()` which returns the value of the `numberAvailable` attribute.

The second point relates to another general decision made in *Unit 7* – that the user interface will be provided with a way of accessing the core objects it needs, so that it can provide objects as arguments to coordinating methods.

However, the *Get Number Available* use case is different in that, as point 3 above notes, there is no coordinating method and so no arguments are involved! All that is needed is for the user interface to obtain a reference to the appropriate `Film` object.

This means that for this use case a coordinating method does not have to be designed. This may seem unusual at first. However, with our approach to communication between user interface and core system this will happen with any use case that simply requires the value(s) of some attribute(s) of a single object to be retrieved and does not require any other services from the core system.

The sequence diagram in Figure 31 shows the message passing involved for a typical *Get Number Available* scenario.

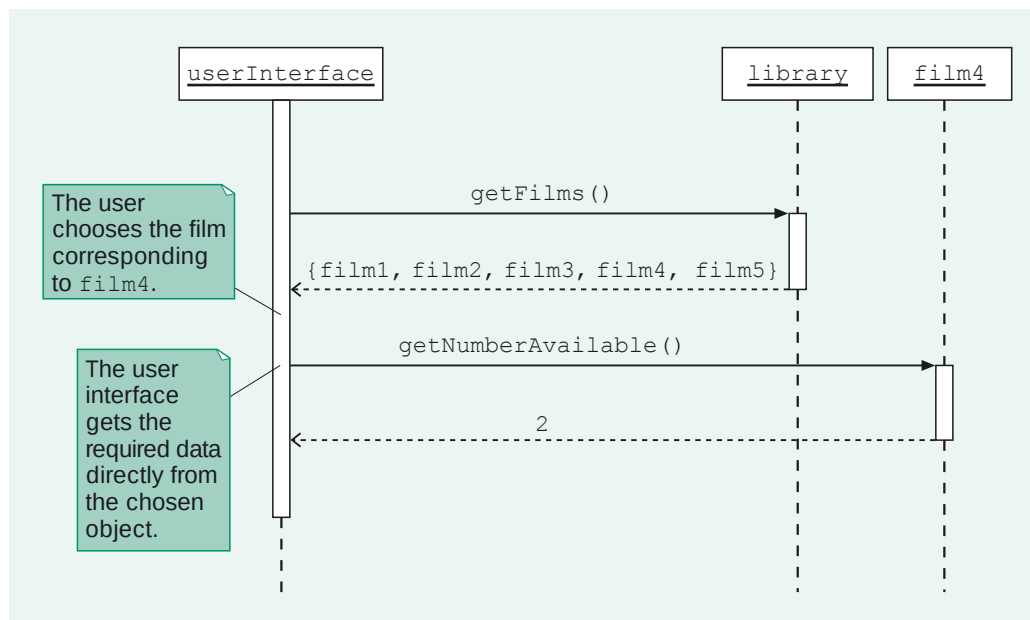


Figure 31 Sequence diagram for *Get Number Available*

From the above discussion it may appear that the *Get Number Available* use case is trivial – there is no walk-through and there are no consequences for the structural model other than the provision of two methods (`getNumberAvailable()` and `getFilms()`), which would have been provided anyway.

However there is a little more to it, because `numberAvailable` is a derived attribute and it is necessary to decide whether or not it should be implemented as an instance variable. We therefore consider briefly the consequences of not implementing `numberAvailable` as an instance variable of `Film`.

If `numberAvailable` was not implemented as an instance variable of `Film`, the designs for *Return DVD* and *Borrow DVD* constructed in Sections 3 and 4 would be simplified as there would be no requirement to update `numberAvailable`. There would be no need in either use case to access a `Film` object or to send any messages to it.

However, there is a price to pay for these simplifications. The `getNumberAvailable()` method in `Film` will not simply return the value of an instance variable. Instead it will need to use the invariant that provides the derivation of the `numberAvailable` attribute in order to calculate its value.

The invariant that allows `numberAvailable` to be derived is the following.

- 1 For any `Film` object, the value of its `numberAvailable` attribute is equal to the number of `DVD` objects it is linked to which are not linked to a `Loan` object.

To see how the system could calculate the value of a `numberAvailable` attribute, consider the scenario illustrated by the object diagram in Figure 32, which shows all the `DVD` objects linked to `film4`, and the relevant `Loan` and `Member` objects.

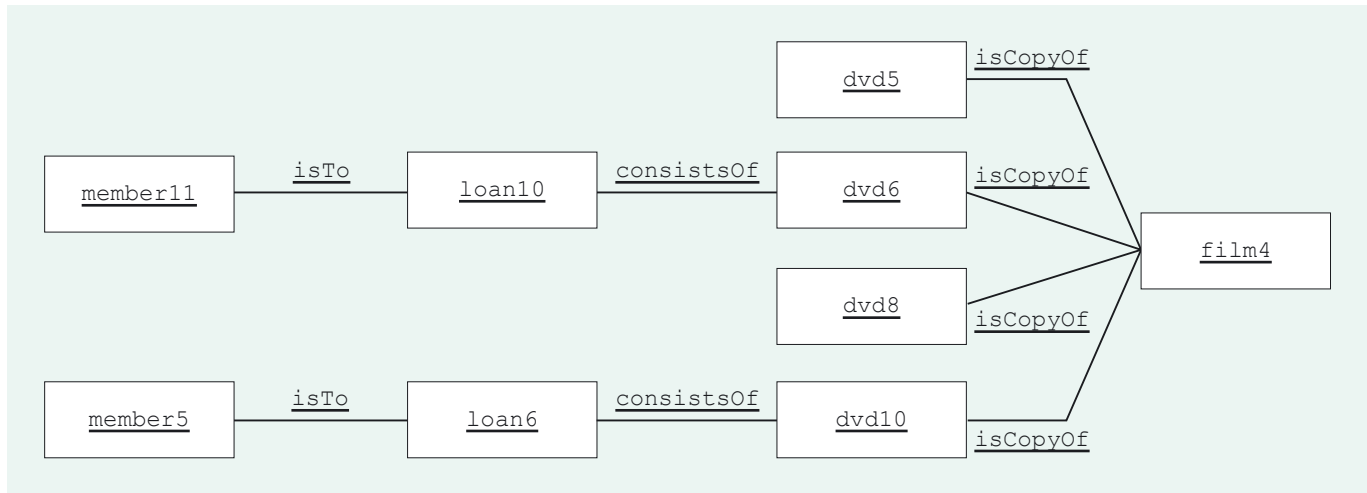


Figure 32 An object diagram showing the DVD objects linked to film4

SAQ 13

Calculate the value of the `numberAvailable` attribute of `film4` given the state shown in the object diagram in Figure 32.

ANSWER.....

The value of the `numberAvailable` attribute of `film4` is 2 – there are exactly two DVD objects that are linked to `film4`, but not linked to any `Loan` object.

In SAQ 13 you calculated the value of the `numberAvailable` attribute in a particular situation. But how might the system perform this calculation? If `film4` can access all the linked DVD objects, it can check each linked DVD object in turn to determine whether it is on loan. To do this would require the `isCopyOf` association to be navigated from `Film` to `DVD`, which would be an additional navigation to those already identified. Furthermore the DVD class would need a method which returns whether the DVD is on loan or not.

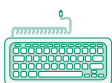
Because of this additional complexity we will implement the `numberAvailable` attribute as an instance variable, even though this means slightly more work for the *Borrow DVD* and *Return DVD* use cases.

5.3 Putting the designs to work

You will now explore a working scenario for the *Borrow DVD* use case.

A working scenario for *Borrow DVD*

In the following exercise you will trace how the design for this use case has been implemented in the code for the methods and constructor. You should also gain an insight into how multiplicities can affect the way that link references are implemented.



Exercise 15

In NetBeans, open the project `Borrow`.

As with the working scenario for *List Films*, the project has two packages, `borrow_core` and `borrow_run`. `borrow_core` (the core system package) contains a partial implementation of the DVD Library System corresponding to our design for the *Borrow DVD* use case. `borrow_run` sets up the objects of the scenario and initiates the use case.

Open the source code for the class `Main` in the package `borrow_run` and read the code. You will find the following sections.

Part 1 – sets up the ‘before’ scenario, following Figure 24, and sets some example attribute values (for simplicity, `numberAvailable` of `film4` is initially set to 1). The number of DVDs of `film4` that are available before the coordinating message is sent is also output.

Part 2 – creates the coordinating object, `library`.

Part 3 – sends the following coordinating message to `library`:

```
borrowDVD(dvd6, member11);
```

When the coordinating message has been sent, the scenario should have changed to the ‘after’ version shown in Figure 25. But how can you tell if it has worked correctly?

In Part 1 `member11` was created but it was not linked to anything at that point. If the use case has worked successfully, `member11` will now be recorded as having one film on loan, whose title is ‘Casablanca’. This can easily be tested, because there is already a method `getLoansAndFilms()`, the coordinating method for the *List Films* use case, written precisely so that the films which a member has on loan can be listed!

Part 4 – invokes this method on `library` and then displays the results.

- Run the project and check that the output is what you expect.
- Next open the class `Loan` in the `borrow_core` package and read its constructor. Identify the two lines of code that create the `isTo` link with the `Member` argument and the two lines of code that create the `consistsOf` link with the `DVD` argument. What else does the constructor do?
- Open the classes `Member` and `DVD` and compare how they reference `Loan` objects and how references to a new `Loan` object are created. What is the reason for the difference? When a `DVD` object creates a reference to a `Loan` object what else does it do?

Discussion.....

- The output should be as follows

```
At start:
Number of DVDs of film4 not on loan = 1
```

```
After borrow:
member11 has the following loans
Film: Casablanca
Return date: 29/01/07
Number of DVDs of film4 not on loan = 0
```

The return date, of course, should be three days after the date on which you run the project.

- The code that creates the `isTo` link between the `Loan` object and the `Member` object is

```
member = aMember;
aMember.addLoan(this);
```

The code that creates the `consistsOf` link between the `Loan` object and the `DVD` object is

```
dvd = aDVD;
aDVD.setLoan(this);
```

The constructor also sets the `returnDate` to the issue date (i.e. the current date) plus 3 days.

If you inspect the coordinating class, `LibCoord`, you will see that it has no `getFilms()` method, which we indicated as necessary for the *Get Number Available* use case. This is because in our simple working scenario, the `Film` object involved is created as part of setting up the ‘before’ scenario, and so does not have to be retrieved from the core system.

- (c) `Member` objects reference `Loan` objects using the instance variable `loans`. This is initialised, in the `Member` constructor, to reference an empty `HashSet` object, intended to contain `Loan` objects. The reference to a new `Loan` object is created by the code

```
loans.add(aLoan);
```

in the method `addLoan(aLoan)`.

`DVD` objects reference `Loan` objects using the instance variable `loan`. This is intended to reference an object of class `Loan` but is initially set to `null`. The reference to the new `Loan` object is created by the code

```
loan = aLoan;
```

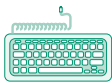
in the method `setLoan(aLoan)`.

The reason for the difference is that the `isTo` association between `Member` and `Loan` has a multiplicity of `0..6` at the `Loan` end, so a `Member` object can have links to several `Loan` objects. On the other hand a `DVD` object can be linked to, at most, one `Loan` object (the multiplicity of `consistsOf` is `0..1` at the `Loan` end), so a collection is not required to hold the reference.

The `DVD` method `setLoan(aLoan)` which creates the reference to a new `Loan` object also sends the message `decrementNumberAvailable()` to `film`, the `Film` object to which the `DVD` object is linked. This ensures that the `Film` object's `numberAvailable` attribute is updated to take account of the fact that a `DVD` of the film has been borrowed.

A working scenario for *Return DVD*

The final exercise implements a working scenario for the *Return DVD* use case.



Exercise 16

In NetBeans, open the project `Return`. As before, the project consists of two packages – in this case, `return.core` and `return_run`.

Open the source code for the class `Main` in the package `return_run` and inspect the code. You will find the following parts.

Part 1 – creates the ‘before’ state of the scenario for this use case (following Figure 15, but with `numberAvailable` set to 0) by using essentially the same code as in Parts 1, 2, 3 and 4 of the `main()` method for the `Borrow` project you met in Exercise 15. In other words:

it creates a `Film` object, a `Member` object and a `DVD` object referenced by `film4`, `member11` and `dvd6` respectively;

it creates the coordinating object, `library`;

it creates a new `Loan` object by sending the message `borrowDVD(dvd6, member11)` to `library` (something has to be borrowed before it can be returned!);

it tests that the loan has been recorded.

Part 2 – causes the borrowed DVD to be returned by sending the following coordinating message to `library`:

```
returnDVD(dvd6);
```

Part 3 – checks the links again and expects to find they have been removed.

- (a) Run the program to check that the output is as expected.
- (b) In the package `return_core` open the class `DVD` and examine the code for its `cancelLoan()` method. You will see that, as shown in the sequence diagram in Figure 22, the `DVD` object forwards the request to its linked `Loan` object by sending the message `removeLoan()`.
How is a `DVD` object's reference to a `Loan` object removed?
- (c) Open the class `Loan` and examine the code for its `removeLoan()` method. Observe the message that is sent to the linked `Member` object instructing it to remove its reference to this `Loan`.
Now look at the code for the corresponding method in `Member`. How is a `Member` object's reference to a `Loan` object removed?
- (d) The *Return DVD* use case involves removing a `Loan` object from the system, as can be seen from the 'before' and 'after' states in Figures 15 and 16.
Why is there no code, in any class, concerned with destroying a `Loan` object?

Discussion

- (a) You should have obtained output like this.

```
At start:
Number of DVDs of film4 not on loan = 1

After borrow:
member11 has 1 current loan(s)
member11 is linked to [a Loan object]
dvd6 is linked to a Loan object
Number of DVDs of film4 not on loan = 0

After return:
member11 has 0 current loans
member11 is linked to []
dvd6 is linked to null
Number of DVDs of film4 not on loan = 1
```

This demonstrates that the links have been successfully removed.

- (b) To remove a `DVD` object's reference to a `Loan` object the `loan` instance variable is set to `null`. A `DVD` object can be linked to, at most, one `Loan` object so the reference is implemented by an instance variable of type `Loan` which is set to `null` when the `DVD` is not on loan.
- (c) The message that is sent to the `Member` object is `removeLoan(this)`.
In the `Member` method `removeLoan(aLoan)` the `Member` object's reference to the `Loan` object is removed with the code `loans.remove(aLoan);`. This removes the reference to the `Loan` object from the collection referenced by the `loans` instance variable.
- (d) No code is necessary for a `Loan` object to be destroyed because in Java when an object has no references the automatic process of garbage collection destroys the object. Hence, having removed the references to the `Loan` object held by the relevant `Member` and `DVD` objects, no references to the `Loan` object remain and thus it will automatically be removed.

In this section, a first design of the DVD Library System has been completed, by investigating the final use case, *Get Number Available*, and exploring typical working scenarios for *Borrow DVD* and *Return DVD*.

Future design iterations would involve completing the designs for these use cases by dealing with alternative and invalid scenarios and considering communication with the user interface. In addition other possible ways of implementing the designs would be considered, using some of the ideas that you are about to meet in *Units 9* and *10*.

6

Summary

In this unit you have applied to the DVD Library System, and seen extended, some of the ideas of *Units 6* and *7*. In particular you have seen the five-stage design process that was introduced in *Unit 6* being used in a different context.

Starting with an initial structural model, a walk-through and sequence diagram were developed for three of the use cases, and some of the implications for the structural model were noted. It was assumed that all pre-conditions of the use cases were met, the so-called typical scenarios.

One important difference with the earlier units is that here different designs for parts of the *List Films* use case were considered at an early stage, rather than after all the other use cases had been dealt with. This is more in accord with the way development would proceed in a real project.

You saw that where objects and links have to be removed it is useful, as with object and link creation, to draw a pair of object diagrams, one to show the position before and one the position after the use case. This helps identify what needs to be done.

You learnt what is involved in removing a link, and how the actions required can depend on the multiplicities and the navigations of the relevant association. You also learnt that when Java is the target language there is no need for an explicit design step corresponding to object destruction – so long as all references to an object are removed, the automatic process of garbage collection will destroy the object.

The final use case, *Get Number Available*, involved a derived attribute and you were reminded of what is involved in deciding whether or not to implement derived attributes. You also learnt that no coordinating method is needed for a use case that simply requires the value(s) of some attribute(s) of a single object to be retrieved.

As you participated in the design process you explored a number of working scenarios – mock-ups of code which implement the use cases in a very basic way, for specific scenarios. These are useful for trying out the designs and looking for possible problems, as well as helping the developer get a rough idea of what the eventual implementation might look like.

LEARNING OUTCOMES

After studying this unit you should be able to create designs for use cases, using walk-throughs and sequence diagrams with increased confidence. (In the following detailed list of outcomes only the asterisked items involve outcomes not addressed previously in *Units 6 and 7*.)

In particular, you should be able to:

- ▶ describe and use each of this unit's key terms (summarised in the Glossary);
- ▶ divide, appropriately, the responsibility for a use case between the user interface and the core system;
- ▶ use a coordinating object to facilitate communication between the user interface and the core system, and to coordinate actions within the core system;
- ▶ construct a typical scenario for a use case, illustrating it using object diagram(s);
- ▶ specify the coordinating method (or methods) for a use case;
- ▶ specify, as appropriate, a pre-condition for a coordinating method;
- ▶ construct a walk-through and a sequence diagram for a typical scenario of a use case;
- ▶ *determine what needs to be done to create or remove a link;
- ▶ *allocate responsibility for creating or removing a link to an appropriate object;
- ▶ compare different designs for a use case using the design principles introduced in previous units;
- ▶ *relate a design to a possible working implementation of a particular scenario;
- ▶ *appreciate that the way a link is implemented can depend on the multiplicity of the association;
- ▶ specify the responsibilities for holding link references, the methods, and navigations required in the structural model as a consequence of a design expressed in a particular dynamic model;
- ▶ identify the consequences, on a given design, of not implementing a derived attribute.

Glossary

working scenario Code that sets up a typical set of objects and implements the design for a use case in a very basic way as a 'proof of concept'.

Index

- A
 - association
 - one-way 20
 - two-way 20, 26
- C
 - cohesion 12
- coordinating
 - class 8
 - message 9, 18
 - method 9, 11
 - object 9–10, 12, 18
- core system 8
- D
 - derived attribute 39, 41
- dynamic model 9
- I
 - implementation model 15
 - invariant 8, 29, 36, 39
- L
 - link
 - one-way 20
 - two-way 20, 22, 27, 35
 - link creation 38
 - link removal 23
- N
 - navigation 15, 42
- O
 - one-way
 - association 20
 - link 20
- P
 - pre-condition 24, 33
- S
 - scenario 9, 17, 32
 - sequence diagram 9
- T
 - two-way
 - association 20, 26
 - link 20, 22, 27, 35
- U
 - user interface 8
- W
 - walk-through 10
 - working scenario 5, 39